

Guia de estilos para Java.

Versión del documento 0.1.0

Propósito

Este documento pretende ser una recopilación de recomendaciones y buenas prácticas al momento de trabajar bajo el lenguaje de programación Java. En concreto se usa como base para la siguiente guía la versión de Java OpenJDK 21, pero en líneas generales, lo que aquí se expone debería ser aplicable a versiones anteriores y posteriores.

Este documento tiene una segunda intención: Ayudarte a mejorar la calidad y velocidad de tu trabajo.

Desde los años 90's y hasta la fecha, gente muy obsesionada con la optimización de procesos (managers e ingenieros industriales) han descubierto una y otra vez que los desarrolladores de software pasan el 20% de su tiempo escribiendo código y un 80% leyendolo. Lo interesante es que a lo largo de las últimas 4 décadas y a pesar de que los desarrolladores han cambiado de lenguajes, IDEs, metodologías, paradigmas y arquitecturas varias veces; esta proporción se ha mantenido más o menos sin cambios.

Por esto es más que razonable dedicar un poco de tiempo y atención a escribir código que sea lo más sencillo de leer y comprender posible.

Este documento tiene una tercera intención: Ser un acuerdo grupal. Estos acuerdos también ayudan a normalizar el código independientemente de quien o cuantas personas trabajen en él, lo que reduce la "variación de jergas" dentro del lenguaje favoreciendo una vez más, la legibilidad del código.

Si hay algo dentro de este documento que no tenga sentido para ti, debes tomarte como una misión personal el que sea corregido.

Lo que este documento no es

Este documento no es una lista de reglas a seguir ciegamente: Este documento no aspira a sustituir, ni menoscabar la capacidad juicio de un desarrollador profesional y competente como puede ser la persona que está leyendo esto. Más que una serie de reglas, considera esto como una oportunidad para abrir la conversación a cerca del estado del arte del lenguaje.

Este documento no es ese martillo que te hace ver a todo el mundo como un clavo. Es perfectamente normal que, mientras escribes código, termines encontrandote con "excepciones a la regla" y dependerá de ti, como el profesional que eres, discernir cuando dicha situación sea, dicha situación.

Este documento no está escrito en piedra. Conforme el equipo de trabajo implementa las recomendaciones de este documento, se encontrarán eventualmente con situaciones que les hagan reevaluar, modificar, omitir y agregar tantas ideas como vean necesarias.

Consistencia

En caso de duda la consistencia es la clave. Si en esta guía se ofrecen múltiples alternativas al momento de aplicar cierto estilo a un fragmento de código, o si un estilo no te convence entonces elige. Pero se consistente en todo el código de tu proyecto.

Solo con ser consistentes, es posible reducir la cantidad de documentación y entrenamiento necesario para entender el sistema. La documentación puede ser más concisa, y una vez que un programador ha aprendido un programa o un módulo del mismo, puede intuir el resto del sistema.

Simplicidad (principio KISS)

Prioriza la simplicidad. Evita tanto ambigüedades de interpretación como código que sobreexplique su intención de uso.

Si existen muchas formas o estilos de escribir una solución a un problema, elige:

- La forma más minimalista de expresarla, siempre y cuando no afecte a...
- La forma más segregada de expresarla, siempre y cuando no afecte a...
- La forma más clara de leerla, siempre y cuando...
- La eficiencia (en ciclos de computo, tamaño del programa compilado, consumo de memoria u otro criterio) no sea mucho más importante que la legibilidad de dicho problema.

Idioma

El código se escribe en inglés.

Las palabras reservadas del lenguaje, las APIs nativas y toda biblioteca que hayas importado de maven están escritas en inglés. Ya que Java es un lenguaje de alto nivel, tiene como objetivo acercarse a un nivel cercano al lenguaje natural humano. Por lo tanto, es necesario homogenizar el idioma pues estar intercalando palabras en inglés y español constantemente en la misma oración/instrucción, interrumpe el flujo natural de la lectura del código.

Existe una segunda razón. Si en algún momento necesitas colaborar con un desarrollador no hispanohablante, tendrá los mismos problemas que tendrías tú para entender el código de, por ejemplo, un francés.

Si una variable o clase representa a una fuente de datos como puede ser una base de datos, un archivo scv o el resultado de una llamada a un API REST, y estos valores ya vienen en español o incluso como acronimos o abreviaciones; es justificable (más no obligatorio) el preservar el nombre, en especial si no tenemos control ni responsabilidad sobre dicha fuente de datos.

Encoding

Usa encoding UTF-8 en todos tus archivos de código.

Declaración de valores

Solo se declara una variable por instrucción a la vez.

```
// correcto
int total;
int average = 0;

// incorrecto
int total, average = 0;
```

Uso de **var**

La palabra reservada **var** se usa para definir una variable dentro de un método como alternativa a especificar el tipo de forma explícita. En vez de esto último, **var** delega al compilador la responsabilidad de inferir el tipo a través de el tipo del objeto que se le está asignando.

```
// en valores primitivos o pseudo primitivos la diferencia es supérflua
String clientName = "Julio Cortázar";
var clientName = "Julio Cortázar";

// en instancias puede ayudar a hacer el código menos redundante
Client client = new Client();
var client = new Client();

// sin embargo, a veces el copilador puede inferir un tipo distinto al deseado.
long totalTickets = 112; // <-- este es long
var totalTickets = 112;  // <-- este es int
```

var puede ayudar a simplificar la lectura de una instrucción de código al minimizar la redundancia de tipos tanto en la referencia como en el objeto a ser asignado. También, indirectamente incentiva al desarrollador a ser más correcto al momento de elegir el nombre de sus variables.

Los casos de uso ideales para **var** es en la creación de objetos de bibliotecas populares, ya que estas están debidamente documentadas, además de que los tipos devueltos por los mismos son ya esperados y reconocibles por cualquier programador habituado al ecosistema del lenguaje.

```
// original
Connection dbConnection = DriverManager.getConnection(DB_URL, USER, PASS);
Statement statement = dbConnection.createStatement();
ResultSet result = statement.executeQuery(query);

// mejor, estas clases son tan comunes que resulta innecesario especificarlas
var dbConnection = DriverManager.getConnection(DB_URL, USER, PASS);
var statement = dbConnection.createStatement();
var result = statement.executeQuery(query);
```

Debe evitarse el uso de **var** cuando el tipo del objeto sea más específico de lo que realmente sea necesario, o lo que es lo mismo, es preferible que nuestra referencia sea de un tipo más general que el que tiene el objeto que estamos asignando.

```
// incorrecto, el tipo de `users` es demasiado específico
var users = new ArrayList<User>();

// correcto, su tipo es una interfaz de propósito más general
List<Users> users = new ArrayList<>();
```

También debe evitarse usar **var** cuando el tipo del objeto a asignar no puede ser inferido a simple vista por el programador al estar abstraído por un builder.

```
// incorrecto, no está claro cual es el tipo de `weeklyReport`
var weeklyReport = Report
    .of(SalesReports.class)
    .combineWith(ManagerPermission.class)
    .from(data)
    .generate();

// correcto, no necesitamos leer el lado derecho de la asignación para saber el tipo de `weeklyReport`
Document weeklyReport = Report
    .of(SalesReport.class)
    .combineWith(ManagerPermission.class)
    .from(data)
    .generate();
```

Uso de **final**

La palabra reservada **final** se usa para indicar que un atributo o variable local en una clase o un método es inmutable.

En el caso de los valores primitivos, se entiende por inmutable, que una vez asignado un valor, este no puede cambiar.

En el caso de objetos, se entiende por inmutable, que una vez asignado un objeto a una referencia, dicha referencia no puede volver a ser reasignada a un objeto distinto.

Esto último no limita en ningún modo la mutabilidad de los atributos de dicho objeto.

```
// La variable clients no puede ser reasignada, pero dependerá de la clase ArrayList
// si sus valores internos cambian.
final List<Client> clients = new ArrayList<>();
clients.add(new Client());
```

Tampoco restringe en modo alguno que un objeto asignado a una variable marcada como **final**

pueda ser referenciado por otra variable no **final**.

```
// Las tres variables apuntan al mismo objeto. La única diferencia es que
// immutableSchedule no puede ser reasignado.
var schedule = new Schedule();
final var immutableSchedule = schedule;
Schedule sameSchedule = immutableSchedule;
```

Todo aquel atributo dentro de tu clase que no cambie debe ser marcado como **final**. Y es una buena práctica diseñar tu clase con la menor cantidad de atributos mutables.

Recuerda que un atributo **final** debe estar inicializado desde su declaración, o si no, debe ser asignado en cada uno de los constructores de tu clase. Esto último es útil cuando el valor de un atributo es desconocido hasta el momento de instanciar la clase.

```
class ProductResource {
    // Correcto, `formatter` está inicializado desde su declaración.
    private final Formatter formatter = new Formatter();
    private final Connection connection;

    public ProductResource(Connection connection) {
        // También correcto, `connection` se define hasta el momento que se instancia.
        this.connection = connection;
    }
}
```

En lenguajes modernos como Scala, Kotlin, o incluso JavaScript; se consigue al mismo tiempo la declaración e inmutabilidad de un valor usando la palabra reservada **val** para los primeros dos, y **const** para el caso de JavaScript.

```
// val en Scala garantiza inmutabilidad
def getSupervisorsName(shop: Shop): String = {
    val metadata = shop.getMetadata
    val supervisor = metadata.getSupervisor
    supervisor.getFullName
}
```

Aunque es recomendable hacer esto en Java, implementar un código equivalente puede sobre saturar de código una definición de una variable local. Pero por otra parte añade información a otros desarrolladores de que cierto valor no va a cambiar a lo largo de su ciclo de vida. Usa cualquiera de las dos alternativas pero se consistente.

```
// `final` añade información sobre la intención de uso de un valor, pero también puede
// entorpecer la lectura por sobre saturación
public String getSupervisorsName(Shop shop) {
    final ShopMetadata metadata = shop.getMetadata();
```

```

    final Employee supervisor = metadata.getSupervisor();
    return supervisor.getFullName();
}

// usar `var` puede ayudar pero aún así sigue saturando
public String getSupervisorsName(Shop shop) {
    final var metadata = shop.getMetadata();
    final var supervisor = metadata.getSupervisor();
    return supervisor.getFullName();
}

```

Variables globales

Evita en la medida de lo posible el uso de variables globales pues todas las partes de tu código que hagan referencia a ella (independientemente de si es para escritura o solo lectura) tendrán dependencia entre ellas (alto acoplamiento).

```

public class Session {
    public static User USER;
}

public void login(String email, String password) {
    // ...
    // ¿Estamos seguros que este es el único lugar en el código donde esta variable
    // es modificada?
    Session.USER = new User();
    // ...
}

// incorrecto: este método solo sirve para validar la sesion actual
public boolean canAccess() {
    return Session.USER.isAdmin();
}

// mejor: este método está desacoplado de la sesion, con lo cual, puede ser
// reutilizado
// en otros contextos
public boolean canAccess(User user) {
    return user.isAdmin();
}

canAccess(Session.USER); // aún hay que refactorizar la variable global

```

No confundir variables globales con constantes. En Java, se entiende por constante todo aquel valor primitivo que es estático y final.

```

public class WarningMessage {
    // por definición es inmutable, y por lo tanto, determinista.
    public static final String INSUFICIENT_MEMORY = "No hay memoria suficiente para

```

```
realizar la operación";  
}
```

Los singletons también son variables globales. Asegurate de que sean mutables solo en la medida de lo indispensable.

```
User user = Session.getInstance().getUser();  
// ¿estamos seguros que esto no rompe otra parte del sistema?  
user.setName("Umberto Eco");
```

```
// `BluetoothAdapter` es la única parte del código que sabe cuales dispositivos  
// hay cerca, por lo tanto es seguro y determinista utilizar esta clase.  
List<BluetoothDevice> devices = BluetoothAdapter.getDefaultAdapter().  
getBondedDevices();
```

Scope

Límita el ciclo de vida de tus variables locales al scope más reducido posible. Esto reduce incertidumbre respecto a donde y cuando una variable pueda llegar a cambiar de valor. Esto también ayuda a hacer énfasis sobre la intención de uso de dichas variables al no dar margen a la libre interpretación.

Estructura

Programación declarativa VS programación imperativa

La programación imperativa es una tendencia a la hora de programar que gira en torno a describir cómo implementar una solución, dando como resultado un código con una tendencia natural a degenerar en código espagueti.

La programación declarativa se enfoca en describir qué es lo que se quiere obtener, dando como resultado una lógica más concisa y un código más desacoplado.

El enfoque declarativo hace que el código sea más fácil de leer y modificar, pero no siempre es la opción más eficiente en términos de rendimiento.

Favorece el enfoque declarativo como el estilo default de abordar los problemas y el imperativo cuando el rendimiento de un fragmento de código sea más relevante que su lectura o incluso que su mantenimiento.

Stream y foreach para iterar arrays y colecciones

Para la mayoría de las iteraciones sobre colecciones, prefiere los métodos del API **Stream** (**map**, **filter**, **reduce**, etc.) y el bucle **for-each**.

Usa **forEach** solo cuando el objetivo sea ejecutar una acción tangencial al stream (como imprimir o

enviar a un log) y no necesites transformar los datos. En cualquier otro caso, prefiere operaciones de `Stream` que devuelvan un resultado.

for para iteraciones de arrays más complejas

Usa `for` cuando necesites:

- Acceder al índice del elemento.
- Mutar el array/colección durante la iteración.
- Iterar sobre múltiples colecciones en paralelo.
- Un control muy específico del flujo (saltos con `break` o `continue`, o steps mayores a 1 o incluso retrocesos en la iteración).

while y recursión para casos más meticulosos

Usa `while` cuando el número de iteraciones no se conoce de antemano y depende de una condición que puede cambiar durante la ejecución.

Como alternativa al `while` puedes usar recursión si el resultado es más fácil de leer. En Java, la recursión no está optimizada por defecto (sin `tail recursion`), así que evita profundidades muy grandes.

Una instrucción por línea

Cada instrucción debe ocupar una línea separada.

Evita usar `this` cuando no se necesita

Usa `this` únicamente cuando sea necesario para desambiguar entre un parámetro y un atributo de instancia con el mismo nombre. En cualquier otro caso, es innecesario y añade ruido al código.

Muchos IDEs pueden ocultar este ruido mediante el resaltado sintáctico, pero el código fuente no debería depender de herramientas externas para cumplir con las buenas prácticas de desarrollo (como mucho, debe apoyarse en ellas).

En constructores y setters, es habitual usar `this` para diferenciar parámetros de atributos. Pero en el resto del código, confía en que el compilador y el lector reconocerán el ámbito de la variable.

Espacios en blanco

Espacios a la derecha

Elimina todos los espacios en blanco a la derecha cuando sean el último carácter de la línea.

La mayoría de IDEs modernos ya incluyen la opción para eliminar dichos espacios automáticamente al guardar cambios.

Un solo espacio en blanco de separación

Usa un solo espacio en blanco para separar dos tokens y solo cuando dichos tokens necesiten ser separados ya sea por sintaxis o por que alguna de las reglas de esta guía así lo recomienda.

Solo se permiten dos o más espacios en blanco consecutivos para indicar indentación.

```
// incorrecto
if (age == 18) canAccess( );

// correcto
if (age == 18) canAccess();
```

Separación entre tokens

Con excepción de `this`, `super`, `return`, `continue` y `break`; todas las palabras reservadas del lenguaje llevan un espacio de separación a su derecha siempre y cuando no sean el último token de la línea:

```
package com.example;

import java.util.ArrayList;

try {
    if (entity instanceof Room) return true;
    else throw new NotAvaliableException('Not active anymore.');
```

Todos los operadores binarios del lenguaje deben llevar un espacio en blanco a ambos lados.

```
// incorrecto
var fahrenheit=(celsius*(9/5))+32;
var canAccess=user.isAdmin()||(user.isSupervisor()&&store.getId()==user.getStore().getId());

// correcto
var fahrenheit = (celsius * (9 / 5)) + 32;
var canAccess = user.isAdmin() || (user.isSupervisor() && store.getId() == user.getStore().getId());
```

Los operadores unarios no llevan espacio de separación.

```
// incorrecto
n ++;
flag = ! flag;

// correcto
n++;
flag = !flag;
```

Los paréntesis y corchetes no llevan espacio de separación con respecto a su contenido. Las llaves usadas para definir el contenido de un array si llevan espacio.

```
// incorrecto
var players=new String[ ] {"Alice","Bob","Carl","Daisy","Erika"};
var currentPlayer=players[ 0 ];

// correcto
var players = new String[] { "Alice", "Bob", "Carl", "Daisy", "Erika" };
var currentPlayer = players[0];
```

Si las llaves son usadas para definir un bloque de código, añade un espacio en blanco a la izquierda de la llave de apertura.

```
// incorrecto
public void sendMessage(String message){
    ...
}

// correcto
public void sendMessage(String message) {
    ...
}
```

Las comas y puntos con coma usados para listar argumentos en el caso de los primeros, o sentencias en el caso de los segundos, llevan un espacio en blanco a la derecha siempre y cuando no sean el último token de la línea.

```
// incorrecto
public void publish(User user,String message,MessageType type,Callback onSended){
    ...
}

// incorrecto
for(var i=0;i<MAX_SIZE;i++){
    ...
}
```

```
// correcto
public void publish(User user, String message, MessageType type, Callback onSended) {
    ...
}

// correcto
for (var i = 0; i < MAX_SIZE; i++) {
    ...
}
```

Espacios en blanco VS Tabs

Se usa solo espacios para hacer indentación.

Indentación

Preferentemente usa una separación de 2 espacios para indicar la anidación de un bloque de código. Opcionalmente, puedes usar 4 espacios, pero no es recomendable por la tendencia del lenguaje a extenderse demasiado a lo horizontal.

No uses una indentación mayor a las anteriores (sobre todo, no uses 8 espacios) y una vez decidida la cantidad de espacios, se consistente en todo el proyecto.

Cuando una línea de código se extiende demasiado en horizontal, es necesario listar las partes de dicha línea en vertical.

```
// Si la firma de la clase es demasiado extensa, separarla en varias líneas simplifica
// la lectura
public class LazyDeployer
    extends Deployer
    implements Serializable, Plugable, Device
{
    ...
}

// Igualmente, si la firma de un método es demasiado larga, el camino a seguir es
// poner sus paramedros cada uno en su propia línea y añadiendo una indentación
public boolean assemble(
    Query query,
    SQLStatement statement,
    Row params,
    Callback callback
) {}

// En secuencias de llamadas es demasiado extensa, hay que evitar alcanzar el límite
// de longitud de caracteres por línea colocando cada nueva invocación en una nueva
// línea con el punto al comienzo de la misma.
Arrays
    .asList("Alice", "Bob", "Carl", "Daisy")
```

```
.stream()
.filter(n -> n.startsWith("A"))
.collect(Collectors.toList());
```

Como alternativa, se sugiere que en estos casos la indentación ha de ser el doble, de lo normal para evitar confundir la indentación por desbordamiento de una línea de código, de la indentación por anidación de bloques de código. En el caso de trabajar con 2 espacios, aquí se usarían 4; y en el caso de usar 4, pasaría a 8.

Queda a criterio del desarrollador si es deseable decantarse por esta última opción. Pero recuerda mantener la consistencia en todo el proyecto.

```
// Se usa una indentación de 4 espacios en vez de 2 ya que cada línea es una parte de
// la cadena de llamadas.
List<Customer> customersSorted =
    ((Function<List<Customer>, List<Customer>>) inputCustomers -> {
        // Estas líneas tienen indentación de solo 2 espacios pues representan un scope
        // nuevo
        List<Customer> customersMutables = new ArrayList<>(inputCustomers);
        Collections.shuffle(customersMutables);
        if (customersMutables.size() >= 1)
            customersMutables.get(0).setMarked(true);
        if (customersMutables.size() >= 2)
            customersMutables.get(1).setMarked(true);
        return customersMutables;
    })
    .apply(initialCustomers)
    .stream()
    .sorted(Comparator.comparing(Customer::getFullName))
    .collect(Collectors.toList());
```

Saltos de línea

Una instrucción por línea

Escribe solo una instrucción por línea.

```
// incorrecto
int width = 200, height = 850, weight = 440;

// incorrecto
for (var i = 0; i < MAX_SIZE; i++) if (item[i].isAvailable()) return true;

// correcto
int width = 200;
int height = 850;
int weight = 440;
```

```
// correcto
for (var i = 0; i < MAX_SIZE; i++)
    if (item[i].isAvailable())
        return true;

// también correcto; el if cuyo contenido es solo el retorno de un valor es lo
// suficientemente simple como para verlo como un solo átomo de significado
for (var i = 0; i < MAX_SIZE; i++)
    if (item[i].isAvailable()) return true;
```

Declaraciones de arrays con una lista de valores demasiado grandes se listan en vertical.

```
String[] countries = {
    "Argentina",
    "Brasil",
    "Canadá",
    "Chile",
    "Colombia",
    "España",
    "Estados Unidos",
    "Francia",
    "Italia",
    "Japón",
    "México",
    "Perú",
    "Portugal",
    "Reino Unido",
    "Sudáfrica",
    "Alemania",
    "Australia",
    "China",
    "India",
    "Rusia"
}
```

Límite de caracteres por línea

La recomendación de la guía de estilos de [Google](#) es limitar la longitud de las líneas a 100 caracteres. Tomaremos esto como el ideal a buscar (soft rule) y 120 caracteres como el límite al que no se debe llegar (hard rule).

La mayoría de editores de código e IDEs incluyen la opción para marcar en pantalla dichos límites.

Una pregunta recurrente entre principiantes es ¿Porqué en la era de monitores de tamaños inmensos sigue siendo necesaria esta regla?

Esta es una heurística inercialmente imprecisa pero no deja de tener una base racional.

En las imprentas y grupos editoriales es sabido desde hace más de un siglo que la longitud de una línea afecta directamente a la velocidad de lectura. A día de hoy el consenso en diseño es que la longitud de un bloque de texto debe estar entre los 600px y los 800px, que es lo justo para no depender de tu mirada periférica ni para tener que girar el cuello y que son las principales fuentes de fatiga en la lectura.

En el caso del código en texto monoespaciado, 100 caracteres es una aproximación descente a esa regla.

Otras razones para preferir esta limitante es que en monitores grandes puedes abrir dos archivos de código simultaneamente. Además en las comparaciones de versiones con git pueden verse las dos versiones de un archivo sin problemas en doble columna.

Otra heurística un poco más elaborada pero que reafirma este punto, versa tal que así:

Los humanos leemos cada línea de código en un programa de la misma forma que un interprete o un compilador lo hace; primero con un proceso de analisis léxico, seguido de un analisis sintactico y por último un analisis sémantico (por arriba de esto ya comienzan las abstracciones, casos de uso y dominio del sistema).

Antes de poder decidir si una línea de código es la causante de un bug, primero hay que leer un token a la vez. Entre más larga sea una línea, mas tokens tendrá. A mayor cantidad de tokens por línea más compleja es dicha línea y es más difícil será analizarla.

```
// La primer línea es más larga que la segunda pero también es mucho más simple y en consecuencia
// es más fácil y rápida de leer

// 21 tokens
var monthlySalesByUser = MonthlySalesReportByUser.of(Document.class).asReport().
withId(userId);
// 36 tokens
var quadraticEquation = (-b + Math.sqrt(Math.pow(b, 2) - (4 * a * c))) / (2 * a);
```

A nivel léxico, sintactico y semantico es fácil reconocer si una línea de código tiene "algo sospechoso" cuando tiene pocos tokens. Si este primer analisis no levanta sospechas, el cerebro humano comienza a interpretar el significado de los nombres de las variables en busca de discrepancias en un nivel más alto de abstracción.

Separación entre variables (de instancia y estáticas)

Las constantes y variables no llevan separación por salto de línea. Pero es recomendable añadir una línea para separar las variables que son estáticas y las que son de instancia.

```
// incorrecto
class Interpreter {
    private static final String MINUTES_SYNTAX = "\\*(\\d|[0-5]\\d)|\\*\\/(\\d|[0-5]\\d)";
```

```

    private static final String HOURS_SYNTAX = "\\*(\\d|1\\d|2[0-3])|\\*\\/(\\d|1\\d|2[0-3])";

    private Pattern patern;

    private Rule rule;
}

// correcto
class Interpreter {
    private static final String MINUTES_SYNTAX = "\\*(\\d|[0-5]\\d)|\\*\\/(\\d|[0-5]\\d)";
    private static final String HOURS_SYNTAX = "\\*(\\d|1\\d|2[0-3])|\\*\\/(\\d|1\\d|2[0-3])";

    private Pattern patern;
    private Rule rule;
}

```

Separación de métodos

Los métodos están separados por un único salto de línea.

```

// incorrecto
private User findById(int id) {
    ...
}
private boolean addUser(User user) {
    ...
}
private boolean editUser(User user) {
    ...
}

// incorrecto
private User findById(int id) {
    ...
}

private boolean addUser(User user) {
    ...
}

private boolean editUser(User user) {
    ...
}

```

```
// correcto
private User findById(int id) {
    ...
}

private boolean addUser(User user) {
    ...
}

private boolean editUser(User user) {
    ...
}
```

La llave de apertura del método no lleva salto de línea.

```
// incorrecto
private int calculate()
{
    ...
}

// correcto
private int calculate() {
    ...
}
```

Entre la declaración de la clase y de los métodos no hay separación de salto de línea.

```
// incorrecto
class UserController {

    private User findById(int id) {
        ...
    }

    private boolean addUser(User user) {
        ...
    }

}

// correcto
class UserController {
    private User findById(int id) {
        ...
    }

    private boolean addUser(User user) {
        ...
    }
}
```

```
}  
}
```

Separación de código dentro de métodos

Para la mente humana, la lectura de un segmento de código no es distinta de la lectura de otro tipo de texto. Al usar un salto de línea para separar un programa en varias partes, estamos intentando organizar el algoritmo subyacente en secciones e ideas concretas. Si bien el código es más legible, esto solo trata los síntomas pero ignora el problema raíz: Depende de como se vea, el código está rompiendo, bien el principio de "Single responsibility" de la programación orientada a objetos, o el principio de "función pura" de la programación funcional.

Dicho de otra forma, estamos intentando segmentar/organizar el código por que el método "sabe hacer demasiadas cosas".

La solución es mover cada sección de código a su propio método. Así ganamos la oportunidad de explicitar (dar nombre) al proceso que estamos computando.

También es importante señalar que separar en metodos más pequeños no siempre simplifica el entendimiento del código por si mismo, pues si este mantiene un alto nivel de cohesión con el método principal, entonces los nuevos metodos, por pequeños que sean, no pueden existir fuera de su contexto. Sin embargo, este es un primer paso en la dirección correcta en la búsqueda de un diseño más correcto.

```
// incorrecto, el método está separado por secciones porque es demasiado complejo  
public void processOrder(Order order) {  
    if (order == null)  
        throw new IllegalArgumentException("El pedido no existe");  
    if (order.getItems() == null || order.getItems().isEmpty())  
        throw new IllegalArgumentException("El pedido necesita por lo menos un articulo");  
    if (order.getCustomer() == null)  
        throw new IllegalArgumentException("Se necesita un clienete");  
  
    BigDecimal total = BigDecimal.ZERO;  
    for (Item item : order.getItems())  
        total = total.add(item.getPrice().multiply(BigDecimal.valueOf(item.  
getQuantity())));  
    order.setTotal(total);  
  
    if (total.compareTo(BigDecimal.valueOf(1000)) > 0) {  
        BigDecimal discount = total.multiply(BigDecimal.valueOf(0.10));  
        total = total.subtract(discount);  
        order.setTotal(total);  
    }  
  
    orderRepository.save(order);  
  
    emailService.sendConfirmation(  
        order.getCustomer().getEmail(),
```

```

        "Order " + order.getId() + " processed"
    );
}

// correcto
public void processOrder(Order order) {
    validateOrder(order);
    calculateTotal(order);
    applyDiscountIfApplicable(order);
    saveOrder(order);
    sendConfirmationEmail(order);
}

private void validateOrder(Order order) { ... }
private void calculateTotal(Order order) { ... }
private void applyDiscountIfApplicable(Order order) { ... }
private void saveOrder(Order order) { ... }
private void sendConfirmationEmail(Order order) { ... }

```

Booleanos

true y false no se comparan

Es redundante utilizar operadores de comparación para evaluar una expresión booleana contra `true` o `false`.

```

// incorrecto, es redundante
if (user.isActive() == true) return user;

// correcto
if (user.isActive()) return user;

```

Aislar expresiones complejas en métodos

Si usas una expresión booleana muy compleja en un `if`, muevela a un método aislado, dale un nombre significativo y llámalo en el `if`. Esto permite separar la evaluación de los datos del control de flujo del programa.

```

// incorrecto, es demasiado complejo
if (
    order.getStatus() != OrderStatus.RELEASED
    && (user.isAdmin() || (user.isMannager() && order.getTotal() < LIMIT_PRICE))
)
    ...

// correcto
private boolean canCancelShipment(User user, Order order) {

```

```

var isNotReleased = order.getStatus() != OrderStatus.RELEASED;
var isLowerThanLimit = order.getTotal() < LIMIT_PRICE;
return isNotReleased && (user.isAdmin() || (user.isMannager() && isLowerThanLimit));
}

if (canCancelShipment(user, order))
...

```

Uso de ==

== evalúa si dos variables apuntan a la misma dirección en memoria. Por lo tanto, solo existen tres situaciones en las que se debe usar este operador:

- 1.- Para comparar si dos valores primitivos son iguales.
- 2.- Para comparar si dos referencias apuntan al mismo objeto.
- 3.- Para comparar enums.

```

int waitingTime = 0;
waitingTime == 30000

Customer customer = new Customer();
Customer lastCustomer = customer;
customer == lastCustomer

SignalStatus signal = SignalSatus.WAITING;
signal == SignalSatus.LOADING

```

Uso de equals

Para comparar si dos objetos son "iguales" usa el método `equals`.

En el caso de tus propias clases, sigue las siguientes recomendaciones sobre la implementación de `equals`:

- Verifica si el parametro a evaluar apunta al mismo objeto.
- Verifica que el parametro no sea nulo.
- Verifica que el parametro sea de la misma clase o un subtipo.
- Compara los atributos primitivos primero.
- Compara los atributos de objetos al final.

Recuerda que la "igualdad" de dos objetos es un concepto subjetivo y es una decisión de diseño elegir cuales atributos son los minimos e indispensables a la hora de comparar la igualdad de dos instancias.

```

class Store {

```

```

...

@Override
public boolean equals(Object object) {
    // verificamos si object apunta al mismo objeto
    if (this == object) return true;
    // verificamos que no sea nulo
    if (object == null) return false;
    // verificamos que tengan tipos compatibles
    if (!object instanceof Store) return false;
    Store store = (Store)object;
    if (id != store.getId() || score != store.getScore()) return false;
    return Objects.equals(name, store.getName())
        && Objects.equals(address, store.getAddress())
        && Objects.equals(supervisor, store.getSupervisor());
}
}

```

Strings

Usa `equals` para evaluar String

Usa `equals` en vez de `==` para comparar strings. No importa que uses `interning`; tendrías que remitirte hasta la asignación de la variable o a la llamada explícita de `intern()` para estar seguro de lo que estás haciendo. Con `equals()` no tienes por qué preocuparte por ninguno de estos detalles técnicos.

```

// incorrecto
String title1 = "El barrio del ángel gris";
String title2 = "El barrio del ángel gris";

title1 == title2;

// correcto
String title1 = "El barrio del ángel gris";
String title2 = "El barrio del ángel gris";

title1.equals(title2);

```

El ejemplo anterior funciona incluso si `title2` es nulo pero fallaría si `title1` lo fuese. Si te encuentras en una situación en la que no tienes garantías de que las variables no son nulas, usa el comparador `Objects.equals()`.

```

// incorrecto si no tenemos garantías de que title1 tiene valor
title1.equals(title2);

// correcto

```

```
Objects.equals(title1, title2);

// también correcto pero es más largo
title1 != null && title1.equals(title2);
```

Strings en una sola línea

La declaración de un string debe mantenerse en una sola línea independientemente de su longitud. Esto se permite ya que los editores de código modernos dan un solo color fijo a todo el texto, con lo cual, al leer el código, todo el texto es interpretado por la mente humana como un solo token (fácil de ignorar).

```
// incorrecto, no es necesario romper la línea.
public static final String SUCCESS_MESSAGE = "Los datos se han guardado correctamente.
Ya puedes "
    + "cerrar la pantalla. Este mensaje es un poco más largo de lo necesario, pero es
con fines "
    + "ilustrativos.";

// correcto
public static final String SUCCESS_MESSAGE = "Los datos se han guardado correctamente.
Ya puedes cerrar la pantalla. Este mensaje es un poco más largo de lo necesario, pero
es con fines ilustrativos.";
```

Strings multilinea

Usa la sintaxis `text block`` en aquellos strings cuyo contenido necesite saltos de línea.

```
// incorrecto, no es multilinea. Y de hecho no compila
public static final String QUOTE = ""La vida es un sueño y lo sueños, sueños son."";

// incorrecto, el resultado es una sola línea complicada de leer
public static final String QUERY = "select * "
    + "from schedules as s "
    + "inner join tasks as t "
    + "on s.id = t.shchedule.id "
    + "where t.status = true "
    + "and s.date = now()";

// incorrecto, hay demasiado ruido como para tener una lectura rápida del texto
public static final String QUERY = "select *\n"
    + "from schedules as s\n"
    + "    inner join tasks as t\n"
    + "        on s.id = t.shchedule.id\n"
    + "where t.status = true\n"
    + "    and s.date = now()";

// correcto
```

```
public static final String QUERY = ""
    select *
    from schedules as s
        inner join tasks as t
            on s.id = t.shchedule.id
    where t.status = true
        and s.date = now();"";
```

En el último ejemplo el output quita todos los espacios en blanco a la izquierda desde el comienzo de la línea hasta el primer caracter que no sea un espacio en blanco; en este caso, 4 espacios.

```
-- output
select *
from schedules as s
    inner join tasks as t
        on s.id = t.shchedule.id
where t.status = true
    and s.date = now();
```

Usar concatenación para casos simples

Si necesitas añadir uno o dos valores de forma dinámica a tu string puedes hacerlo con concatenación de strings.

```
// correcto
return "Haz completado " + tasksDone + " tareas de " + totalTasks + ".";

// incorrecto, demasiadas variables complican la lectura del string
return "Bienvenido " + customerName + ", tienes " + newMessages + " mensajes nuevos de " + sender + ", debes responderlos antes de " + date + ".";

// también incorrecto
return "Bienvenido " + customerName + ", tienes " + newMessages + " mensajes nuevos de " + sender
    + ", debes responderlos antes de " + date + ".";
```

Pero debes asegurarte de que solo pongas variables o llamadas de métodos simples, nunca computos complejos.

```
// incorrecto
var hotSiteMessage = "¡Advertencia! ¡El site ha alcanzado una temperatura de " + lang
    .equals("es") ? celsius : ((celsius * 9 / 5) + 32) + "! ¡Llama a mantenimiento inmediatamente!";

// correcto
float temperature;
if (lang.equals("es"))
```

```

    temperature = celsius;
else
    temperature = (celsius * 9 / 5) + 32;

var hotSiteMessage = "¡Advertencia! ¡El site ha alcanzado una temperatura de " +
    temperature + "! ¡Llama a mantenimiento inmediatamente!";

```

Los string son representados en todos los editores de código con un solo color porque tienen la intención de ser considerados como un único token. En el momento en que comienzas a leer el contenido del string pasas de leer un dialecto (Java) a leer otro (español, inglés...). Si tu cerebro se encuentra en modo "lectura de texto humano" y se encuentra de repente con un fragmento de código, tendrá que forzosamente volver a cambiar de modo para calcular qué se va a mostrar en el output y luego tendrás que cambiar una vez más de modo para continuar leyendo el string. Usar una variable con un nombre significativo es la diferencia entre encontrarte en tu lectura con un semáforo en rojo o solo reducir la velocidad.

String.format()

Usa `String.format()` para casos de concatenaciones más complejos.

```

var receiptHeaderFormat = "%-15s %5s %10s\n";
var receiptItemsFormat = "%-15s %5d %10.2f\n";

String header = String.format(receiptFormatHeader, "Artículo", "Cant", "Precio");
String separator = String.format(receiptFormatHeader, "-----", "----", "-----");

var itemList = "";

for (var item : order)
    itemList += String.format(
        receiptItemsFormat,
        item.getName(),
        item.getQuantity(),
        item.getTotal()
    );

System.out.println(header);
System.out.println(separator);
System.out.println(itemList);

```

Números

Usa `_` para mejorar la legibilidad de números grandes

```

// no es incorrecto, pero no es tan fácil de leer
private static final double CREDIT_LIMIT = 10000000;

```



```
// incorrecto
Long counter = Long.valueOf(0);
Float sum = Float.valueOf(0);
Double limit = Double.valueOf(1_000_000);

// incorrecto
Long counter = (long)0;
Float sum = (float)0;
Double sigma = (double)1_000_000;

// correcto
Long counter = 0L;
Float sum = 0F;
Double sigma = 1_000_000D;
```

Usa BigDecimal para evitar perdida de precisión

Los tipos `float` y `double` utilizan representación binaria en coma flotante ([IEEE 754](#)). Esto significa que muchos números decimales (como `0.1`) no pueden representarse exactamente, lo que provoca errores de redondeo acumulativos.

```
// incorrecto, total es igual a 0.30000000000000004
double price = 0.1;
double total = price * 3;

// correcto, total es igual a 0.3
BigDecimal price = new BigDecimal("0.1");
BigDecimal total = price.multiply(BigDecimal.valueOf(3));
```

Nunca uses el constructor `BigDecimal(double)` porque hereda la imprecisión del `double`. Siempre usa el constructor `BigDecimal(String)` o el método `BigDecimal.valueOf(double)`.

```
// incorrecto, el valor es 0.100000000000000005551...
BigDecimal price = new BigDecimal(0.1);

// correcto
BigDecimal price = new BigDecimal("0.1");

// también correcto
BigDecimal price = BigDecimal.valueOf(0.1);
```

Define una escala y política de redondeo

Las operaciones aritméticas con `BigDecimal` deben especificar siempre la escala y el modo de redondeo. No confíes en el redondeo por defecto, ya que puede lanzar una excepción (`ArithmeticException`) en divisiones no exactas.

```
// incorrecto, lanza 'ArithmeticException' si la división no es exacta
BigDecimal result = price1.divide(price2);

// correcto
BigDecimal result = price1.divide(price2, 2, RoundingMode.HALF_EVEN);
```

El `RoundingMode.HALF_EVEN` (redondeo bancario) es el modo predeterminado en muchos estándares financieros y evita sesgos estadísticos. Es la opción recomendada si no hay requisitos específicos.

Utiliza `BigDecimal` como tipo inmutable

`BigDecimal` es inmutable. Cada operación devuelve una nueva instancia en lugar de modificar la existente. Asegúrate de asignar siempre el resultado de las operaciones a una variable.

```
// incorrecto, 'add' no es mutable
BigDecimal total = BigDecimal.ZERO;
total.add(price);

// correcto
BigDecimal total = BigDecimal.ZERO;
total = total.add(price);
```

Compara `BigDecimal` correctamente

No uses `equals()` para comparar `BigDecimal` porque también compara la escala. Usa `compareTo()` que ignora la escala para la comparación de valor numérico.

```
BigDecimal a = new BigDecimal("2.0");
BigDecimal b = new BigDecimal("2.00");

// incorrecto, no son iguales
a.equals(b);

// correcto, solo compara el valor del número
a.compareTo(b) == 0;
```

Para comparar con cero, usa `BigDecimal.ZERO.compareTo(valor) == 0` en lugar de `BigDecimal.ZERO.equals(valor)`.

Usa `BigDecimal` en valores de retorno de APIs

Cuando tu API devuelve valores monetarios o precisos, utiliza siempre `BigDecimal` en lugar de `double` o `float` para garantizar que el consumidor reciba el valor exacto sin pérdida de precisión.

Aunque `BigDecimal` tiene un coste de rendimiento mayor que `double`, este coste es insignificante para la mayoría de las aplicaciones empresariales comparado con el riesgo de errores de precisión.

Para operaciones masivas (estadísticas, big data), considera alternativas como **long** representando fracciones.

Métodos

Menos de 50 líneas de código por método

Límita el número de líneas de código a 50 por método. Esto cumple varias funciones:

- El método puede ser leído de principio a fin sin necesidad de hacer scroll.
- En general, reduce la complejidad del método.
- En concreto, reduce las responsabilidades del método.

No pongas múltiples instrucciones en la misma línea a modo de hack para cumplir esta regla.

```
// incorrecto: la primer línea está sobrecargada de información con el único fin
// de reducir el número de líneas del método
public Model getById(List<Model> items, int id) {
    for (var item : items) if (item.getId() == id) return item;
    return null;
}

// correcto: cada línea es una instrucción clara y concisa
public Model getById(List<Model> items, int id) {
    for (var item : items)
        // Aún siendo dos instrucciones, esta línea sigue siendo una unidad de significado
        if (item.getId() == id) return item;
    return null;
}
```

Si tu método necesita más líneas de código para cumplir su labor, es síntoma de que "sabe hacer demasiadas cosas" segrega tu código en múltiples métodos con pocas responsabilidades. Recuerda que para entender qué es lo que hace un método es necesario entender cada una de sus partes.

```
// incorrecto: hay demasiada información, entender este código toma tiempo.
// hacerle cambios toma aún más.
@PostMapping
public ResponseEntity<?> createRecord(@RequestBody RecordRequest request) {
    if (request.getDate() == null)
        return ResponseEntity.badRequest().body(Map.of("error", "El campo 'date' es obligatorio.));
    int currentYear = Year.now().getValue();
    if (request.getDate().getYear() != currentYear)
        return ResponseEntity.badRequest().body(Map.of("error", "La fecha debe ser del año en curso.));

    String sql = "INSERT INTO records (name, amount, date) VALUES (?, ?, ?)";
```

```

try {
    int rows = jdbcTemplate.update(
        sql,
        request.getName(),
        request.getAmount(),
        java.sql.Date.valueOf(request.getDate())
    );
    if (rows == 1)
        return ResponseEntity.status(HttpStatus.CREATED).body(Map.of("message",
"Registro creado."));
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(Map.of("error", "No se insertó el registro."));
} catch (DataAccessException ex) {
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(Map.of("error", "Error al insertar el registro.", "detail", ex.
getMessage()));
}
}

// correcto: el método describe que es lo que hace y no cómo lo hace,
// esa responsabilidad es delegada a otros métodos privados más especializados
@PostMapping
public ResponseEntity<?> createRecord(@RequestBody RecordRequest request) {
    if (!isValid(request)) return getInvalidDateResponse();
    var result = save(request);
    if (result) return getSuccessResponse();
    return getSaveErrorResponse();
}

private boolean isValid(RecordRequest request) { ... }
private boolean save(RecordRequest request) { ... }
private ResponseEntity<?> getInvalidDateResponse() { ... }
private ResponseEntity<?> getSaveErrorResponse() { ... }
private ResponseEntity<?> getSuccessResponse() { ... }

```

Menos de 10 parámetros

Los constructores son la excepción a la regla.

Reduce el número de parámetros que pasas a tu método lo más posible. Esto se consigue a través de la segregación de las responsabilidades del método tal como se ha mostrado anteriormente; es decir, revisa si es posible depurar tus variables antes de llamar al método en vez de hacerlo dentro.

Si aún así tienes demasiados parámetros es probable que estes obviando una entidad en tu diseño.

```

// incorrecto: 13 parámetros, difícil de leer, de probar y de mantener
public Receipt createOrder(
    String customerId,
    String customerName,

```

```

String customerEmail,
String shipStreet,
String shipCity,
String shipState,
String shipZip,
List<OrderItem> items,
BigDecimal discount,
BigDecimal taxRate,
String paymentMethod,
String currency,
boolean expressShipping
) {
    ...
}

// correcto: Las variables quedan encapsuladas en unidades de significado
// a un nivel más alto de abstracción
public Receipt createOrder(
    Customer customer,
    Address shippingAddress,
    List<OrderItem> items,
    Payment payment,
    OrderOptions options
) {
    ...
}

```

No añadas constantes ni variables de instancia a tu clase solo para cumplir con esta regla. Esto vuelve el uso del método más complicado y la clase completa es más confusa de utilizar.

No mutar parámetros

...excepto si esa es la intención explícita del método.

```

// correcto, el nombre del método ya advierte de una mutación sobre el parámetro
public void markAsConfirmed(Task task) {
    task.setStatus(TaskStatus.CONFIRMED);
    task.setUpdatedAt(Instant.now());
    sendMail(new TaskReport(task));
}

// también correcto
public void appendAction(Task task, Action action) {
    ...
}

appendAction(task, action);

// mejor, es el propio objeto el encargado de mutar su información
class Task {

```

```

public void append(Action action) {
    ...
}
}

task.append(action);

```

Modificar los valores de un parámetro, no como la intención principal del método sino como un efecto secundario añade incertidumbre al usuario de la misma sobre su uso.

Una alternativa bastante común en otros lenguajes es crear versiones inmutables y mutables del mismo método.

No más de 4 niveles de indentación

Mantén el código de tu método con la menor cantidad posible de scopes anidados. Esto ayuda nuevamente a la legibilidad pues cada nueva indentación fuerza a la mente que lee el código a considerar todos los "contextos" que gobiernan a una línea al momento de hacer un cambio.

De nuevo, la solución es segregar el código en múltiples métodos especializados de modo que cada uno de ellos tenga un scope limitado y con pocas anidaciones.

```

public boolean intersects(List<Device> localDevices, List<Device> remoteDevices) {
    for (final var localDevice : localDevices)
        if (!localDevice.isLoaded())
            continue;
        else if (localDevice.isReachable()) {
            var localMac = "";
            try {
                while (!localDevice.isAvaliable())
                    localDevice.connect();
                localMac = localDevice.getMacAddress();
            }
            catch (UnreachableExceptionl error) {
                return false;
            }
            for (final var remoteDevice : remoteDevices)
                if (!remoteDevice.isLoaded())
                    continue;
                else if (remoteDevice.isReachable())
                    try {
                        while (!remoteDevice.isAvaliable())
                            remoteDevice.connect();
                        var remoteMac = remoteDevice.getMacAddress();
                        if (localMac.equals(remoteMac))
                            return true;
                    }
                    catch (UnreachableExceptionl error) {
                        return false;
                    }
            }
        }
}

```

```
}  
return false;  
}
```

Aplica los principios de la programación funcional

En realidad no todos los principios de la programación funcional son aplicables a Java, pero hay algunos que pueden ser beneficiosos:

- Funciones como ciudadanos de primer nivel: esto se puede lograr hasta cierto punto a través de lambdas.
- Tus funciones deben ser deterministas: esto también es deseable en métodos.
- Las funciones no tienen efectos secundarios: no es aplicable a todos los escenarios, pero es algo a tener en cuenta.
- Las funciones no mutan variables: esta no debería ser aplicable pues en OOP un método por definición, muta (o lee) los atributos de la clase. Por otra parte pueden existir clases cuyo propósito es el ser un repositorio de funciones o también llamadas clases helpers (por ejemplo la clase Math); en esos casos el principio debería respetarse.
- Las funciones deben ser declarativas: una vez más, nos remitimos al estilo de programación declarativo como ideal a buscar.
- Recursión para iterar: Java no está tan optimizado en su cola de llamadas, por lo que no es recomendable aplicar recursión en demasiada profundidad.

Clases

Estructura

La firma de tus clases debe seguir la siguiente estructura.

Primero indica la herencia, después la lista de interfaces a implementar.

```
// incorrecto  
class PatientsScheduler implements Serializable, Configurable extends Scheduler {  
    ...  
}  
  
// correcto  
class PatientsScheduler extends Scheduler implements Serializable, Configurable {  
    ...  
}
```

La llave de apertura de definición de la clase debe estar en la misma línea de la firma.

```
// incorrecto  
class Command
```

```
{
    ...
}

// correcto
class Command {
    ...
}
```

Si la firma se extiende demasiado en horizontal, puedes poner las extensiones e implementaciones en líneas a parte con 4 espacios de indentación.

```
// incorrecto
class PatientsScheduler extends Scheduler implements Serializable, Configurable,
Taxable, Deployable, Cronable {
    ...
}

// correcto
class PatientsScheduler
    extends Scheduler
    implements Serializable, Configurable, Taxable, Deployable, Cronable
{
    ...
}
```

No debe haber ninguna línea en blanco de separación entre la declaración de una clase y su contenido.

```
// incorrecto
public class ImageAnalyzer {

    private static final int MAX_WEIGTH = 1024;

    public Report analyze(Image image) {
        ...
    }

}

// correcto
public class ImageAnalyzer {
    private static final int MAX_WEIGTH = 1024;

    public Report analyze(Image image) {
        ...
    }
}
```

El orden de los elementos dentro de una clase son de arriba a abajo:

- 1.- Constantes
- 2.- Variables estáticas
- 3.- Métodos estáticos
- 4.- Valores de instancia
- 5.- Variables de instancia
- 6.- Constructores (del más simple al más elaborado)
- 7.- Getters y setters
- 8.- Métodos de instancia
- 9.- Clases anidadas

Cada una de estas secciones va separada con una línea vacía de separación.

A su vez, cada uno de estos apartados debe estar organizado por sus modificadores de acceso (public, default, protected y private) del más permisivo al más restrictivo.

Dentro de las secciones mencionadas; los métodos estáticos, constructores, getters, setters y métodos de instancia llevan una línea en blanco de separación entre sus elementos. Los demás no llevan espacio de separación.

```
class Pattern {
    public static final int MAX_LENGTH = 300;
    private static final String ERROR_MESSAGE = "Patrón no reconocido";

    private static int patternsTested = 0;

    public static boolean test(String pattern) {
        ...
    }

    private final PatternConfig config = new PatternConfig();
    private final UUID uuid = UUID.randomUUID();

    private String regex;
    private int restrictionLevel = 1;

    private Pattern() {
        ...
    }

    public Pattern(PatternConfig config) {
        ...
    }
}
```

```

public Pattern(PatternConfig config, String regex) {
    ...
}

public Pattern(PatternConfig config, String regex, int restrictionLevel) {
    ...
}

public String getRegex() {
    ...
}

public void setRegex(String value) {
    ...
}

public int getRestrictionLevel() {
    ...
}

public void setRestrictionLevel(int value) {
    ...
}

public boolean match(String text) {
    ...
}

public List<Strng> find(String text) {
    ...
}

private class Rule {
    ...
}
}

```

Ha de notarse que en la sección de constructores tiene más prioridad la complejidad y número de parámetros que el modificador de accesos. En general, la complejidad de las sobrecargas de métodos tiene más relevancia que los modificadores de acceso.

Atributos

Todos los atributos de tu clase deben ser privados. Punto.

Todo atributo que no cambie de valor durante el ciclo de vida de la instancia debe ser marcado como **final**.

Propiedades

Si necesitas acceder a un atributo desde una clase externa hazlo a través de un getter.

Si necesitas modificar un atributo desde una clase externa hazlo a través de un setter.

Si necesitas acceder a un atributo desde una clase heredada hazlo a través de un getter.

Si necesitas modificar un atributo desde una clase heredada hazlo a través de un setter.

Los getters y setters tienen su contenido en una línea a parte de donde se establece su firma.

```
// incorrecto
public int getSize() { return size; }

// correcto
public int getSize() {
    return size;
}
```

Solo crea getters y setters para un atributo si realmente lo necesita. Si el valor de un atributo no necesita ser visible desde otra clase, entonces no necesitas crear el getter. Si el valor de un atributo solo será modificado desde dentro de la misma clase, entonces no necesitas crear el setter.

Si tu atributo no necesita ser modificado después de la creación de la instancia, entonces no necesitas ni el getter ni el setter, y tu atributo necesita ser un valor (final).

Si el valor no va a cambiar desde la compilación, entonces lo que necesitas una constante.

Las propiedades generalmente solo tienen el objetivo de mutar o exponer atributos, pero también pueden realizar otras operaciones intermedias siempre y cuando no sean demasiado complejas ni extensas. Dicho esto, si una propiedad con una operación intermedia es usada de forma exhaustiva, es preferible crear un atributo de caché para hacer dicho cálculo la menor cantidad de veces posible.

```
// correcto
public int getFullName() {
    return name + " " + lastName + " " + motherLastName;
}

// también correcto si 'getFullName' es llamado demasiadas veces
private String fullName;

public void setName(String value) {
    name = value;
    fullName = name + " " + lastName + " " + motherLastName;
}

public void setLastName() { ... }
```

```
public void setMotherLastName() { ... }

public int getFullName() {
    return fullName;
}
```

Métodos

Todo metodo debería interactuar leyendo o modificando por lo menos un atributo de la clase, si no lo hace, debes replantearte si dicho metodo pertenece realmente a la clase.

Una clase por archivo

Define una clase y solo una clase por archivo. Se entiende que una clase anidada es parte de la clase en cuestion.

Interfaz pública

Mantén la interfaz pública de tu clase lo más reducida posible. Todo lo que no necesite ser llamado desde una clase externa debe ser marccado como **private**. Si de estos elementos alguno es nesesario en una clase heredada, entonces nesesita ser como mucho **protected**.

Visto de otra forma, escribe todo tu código privado y luego ve abriendo su acceso según lo vayas necesitando en ambitos externos a tu clase.

Clases anidadas

Existen varias circunstancias bajo las cuales es sensato utilizar clases anidadas. A continuación se enuncian algunas de ellas y también algunas alternativas para evitar sobresaturar de información tus clases. Por ejemplo, para aislar una clase **X** que a nivel semántico y de dominio del sistema no tiene sentido que sea accesible exclusivamente por otra clase **Y**.

```
public class SendMailJob implements Job {
    private Action action = new SendMailAction();

    public void execute() {
        action.start();
    }

    private class SendMailAction implements Action {
        public void start() {
            ...
        }
    }
}
```

Un argumento en contra de esta forma de organizar tus clases es que no solo hay una alta cohesión sino también un alto acoplamiento entre las dos clases. Una forma alternativa de organizar tus

clases es moviéndolas a un package más específico de modo que el subtipo de `Action` siga siendo accesible para el subtipo de `Job` pero no para el resto del sistema. De esta forma volvemos al principio de una sola clase por archivo y no tenemos que añadir una indentación extra en cada elemento de la clase antes anidada.

```
// mejor, creamos un package específico para estas dos clases
package com.example.job.sendmail;

public class SendMailJob implements Job {
    private Action action = new SendMailAction();

    public void execute() {
        action.start();
    }
}

package com.example.job.sendmail;

// al no ser public, esta clase no es accesible fuera del package
class SendMailAction implements Action {
    public void start() {
        ...
    }
}
```

Sobre todo, evita llenar de demasiada información tu clase anidada. Si esta crece demasiado, quizás es momento de replantearte tu diseño.

No defines nunca más de un nivel de anidación de clases.

```
// incorrecto
public class Order {
    ...

    private static class State {
        ...

        public class StateHelper {
            ...
        }
    }
}
```

Por default define tus clases anidadas como estáticas excepto si realmente necesitas acceso a las variables de instancia de la clase huésped. Recuerda que la mayoría de los exploits de códigos son causa de una excesiva ofuscación como para anticipar (o siquiera testear) todos los escenarios de uso posibles.

Clases anónimas

Evita diseñar tus clases con el objetivo de ser implementadas como clases anónimas. También evita implementar clases de terceros como clases anónimas cuando sea posible.

Esto es preferible ya que la implementación de una clase anónima es demasiado engorrosa, tanto de escribir como de leer.

Si necesitas una implementación de una clase con acceso al scope en el que se usa, es preferible utilizar lambdas.

Clases locales

No uses clases locales. Punto.

Métodos estáticos

En general los metodos estáticos son más difíciles de modificar a posterior (recuerda el open close principle)

Favorece el diseño de tus métodos estáticos como funciones, esto es, que no tengan efectos secundarios y que no muten variables estáticas.

No uses métodos estáticos simplemente para no lidiar con OOP.

No accedas a métodos estáticos a través de subtipos ni los combines con herencia. Los métodos estáticos no son polimorficos.

Utiliza `static imports` cuando lo veas posible para simplificar la lectura de tu código.

Variables estáticas

Las variables estáticas, son dentro de Java variables globales. Evita en la medida de lo posible su uso.

Si tienes una razón de peso para el uso de una variable global, asegurate de que sea privada. Y si necesitas modificarla desde un punto externo a la clase en que está declarada, hazlo a través de métodos estáticos en la misma clase.

También considera la opción de usar un singleton en su lugar.

Evita el 'God Object'

El "God Object" es un antipatrón muy común en principiantes en el cual una clase tiende a crecer en tamaño y responsabilidades. De forma natural existen dos clases que tienden a degenerar en este antipatrón:

1.- La clase `User`: Si no prestas atención, esta clase puede terminar encargandose de gestionar demasiadas responsabilidades en el sistema (crud de usuarios, login, cambios de contraseña...).

2.- Aquella clase que sea el centro del dominio de tu proyecto. Si tu aplicación es una app de viajes,

entonces la clase con tendencia a degenerar en un "God Object" es la clase `Travel`; si tu app es un kanban, entonces será la clase `Task`.

Ten siempre en cuenta el principio de single responsibility.

Usa el patrón builder

Al momento de diseñar una clase siempre tienes que tener en mente cómo va a ser usada por otros desarrolladores. Si la inicialización de tus instancias llega a ser un proceso demasiado ambiguo por tener un a gran cantidad de posibles configuraciones prueba a usar el patrón builder para simplificar la construcción de tu clase.

```
// incorrecto
Document document = new Document();

// correcto
Document document = Document
    .builder()
    .setPointer(10, 10)
    .setFontSize(18)
    .setText("Reporte de actividades")
    .setPointer(10, 50)
    .build();
```

Factory method

Cuando la creación de una instancia no es trivial (requiere validaciones, valores por defecto, o lógica condicional), o cuando quieres dar un nombre semántico al proceso de creación, utiliza métodos factory en lugar de invocar directamente el constructor.

Estos métodos deben devolver una instancia de la clase y su nombre debe reflejar claramente qué tipo de objeto están creando (ej. `of`, `from`, `valueOf`, `create`, `newInstance`).

```
// incorrecto: constructor genérico y sin semántica
// no queda claro qué tipo de documento se está creando
Document doc = new Document("informe.pdf", 135, 240, true, false);

// Correcto: factory method con nombre expresivo
Document doc = Document.createReport("informe.pdf");
```

Herencia

La herencia no es para reutilizar código

Cuando defines una nueva entidad en base a otra clase ya existente, esta entidad debe entenderse como un subtipo (**is-a**). Si tu clase no cumple esta relación entonces no deberías usar herencia.

```
// incorrecto, un cajero no es una impresora
class Printer { ... }
class CashMachine extends Printer { ... }

// correcto, una service account es un subtipo de usuario
class User { ... }
class ServiceAccount extends User { ... }
```

Si la razón por la que quieres implementar herencia es porque necesitas una determinada funcionalidad o característica de la clase base y no quieres duplicar código, entonces debes replantearte si dicha funcionalidad realmente pertenece a la clase base en primer lugar. Modificar las preconcepciones que tenemos sobre qué es y que no es parte de nuestras entidades (refactorizar nuestros diseños) es parte natural del ciclo de vida de desarrollo de todo proyecto.

Composición antes que herencia

Debes considerar a la herencia como la última opción disponible. Esto debe ser así pues su implementación es algo que solo se puede hacer una vez y su refactorización resulta complicada debido al alto acoplamiento que genera entre la clase base y la clase heredada.

Diseña tus clases en base a composición antes que en base a herencia. La composición ofrece una mayor flexibilidad con menor acoplamiento, e incluso permite un mayor encapsulamiento de responsabilidades e inyección de dependencias.

La composición se basa en mantener instancias de otras clases en vez de heredar el comportamiento completo, de modo que es más fácil limitar el uso de la clase componente y al mismo tiempo es más fácil expandir la funcionalidad de la clase compuesta a través de múltiples componentes para realizar todas sus tareas.

```
// incorrecto, `CashMachine` no es una impresora con habilidades extra.
class CashMachine extends Printer {
    private void printReceipt() {
        super.print();
    }
}

// correcto, `CashMachine` es una entidad compuesta de muchas entidades.
class CashMachine {
    private final Printer printer = new Printer();
    private final CardAuthenticator authenticator = new CardAuthenticator();
    private final CurrencyCalculator calculator = new CurrencyCalculator();

    private void printReceipt() {
        printer.print();
    }
}
```

Interfaces antes que herencia

Aún si tu entidad responde correctamente a la respuesta "X es un subtipo de Y" eso no necesariamente significa que la herencia sea la mejor opción, justamente por el tema antes mencionado de el alto acoplamiento que habría entre los dos tipos.

Una alternativa a explorar antes, es utilizar el principio de interface segregation para quitar responsabilidades al supertipo Y que quizas no le competen del todo.

```
// correcto, una service account es un subtipo de usuario
class User { ... }
class ServiceAccount extends User { ... }

private void login(User user) { ... }

// mejor, al no ser por definición "humana", una `ServiceAccount` no necesita hacer
// todo lo que
// `User`.
interface Authenticable { ... }
interface Authorizable { ... }
interface Taxable { ... }
interface Addressable { ... }

class User implements Authenticable, Authorizable, Taxable, Addressable {}
class ServiceAccount implements Authenticable, Authorizable {}

private void login(Authenticable prospect) { ... }
```

Evita árboles de jerarquías complejas

Las jerarquías de herencia profundas o extensas son difíciles de entender, modificar y depurar. Cada nivel añade complejidad y aumenta el riesgo de efectos colaterales al modificar la clase base.

Prefiere jerarquías planas con interfaces para comportamientos transversales.

Si encuentras que necesitas muchos niveles, probablemente estés abusando de la herencia y deberías reconsiderar el diseño usando composición e interfaces.

Garantiza Liskov

El principio de Liskov establece que los objetos de una subclase deben poder sustituir a objetos de la superclase sin alterar el comportamiento esperado del programa.

En la práctica, esto significa que la subclase no debe:

- Reforzar las precondiciones (requisitos de entrada) de los métodos de la superclase.
- Lanzar excepciones que no sean declaradas o esperadas por la superclase.
- Cambiar el estado interno de manera que viole las invariantes de la superclase.

Si al crear una subclase te ves obligado a anular muchos métodos y dejar otros sin implementar o lanzando excepciones, es que estás rompiendo el principio y probablemente deberías usar composición o refactorizar tus clases.

Interfaces

Haz segregación de interfaces

Una interfaz debe ser específica y representar una única responsabilidad o rol. Evita crear **fat interfaces** que obliguen a las clases implementadoras a depender de métodos que no necesitan.

Si una interfaz tiene múltiples grupos de métodos que son utilizados por diferentes tipos de clientes, es señal de que debe dividirse en interfaces más pequeñas.

```
// incorrecto, Worker tiene demasiadas responsabilidades
interface Worker {
    void run();
    void log();
    void schedule();
}

// correcto, es más fácil refactorizar
interface Worker {
    void run();
}

interface Loggable {
    void log();
}

interface Schedulable {
    void schedule();
}

class Executable implements Worker, Loggable {}
```

Al segregar interfaces, el código cliente puede depender únicamente de la interfaz que necesita así que también terminas cumpliendo con el principio de inversión de dependencias. Esto facilita las pruebas unitarias y desacopla los módulos.

Composición antes que herencia múltiple

Java permite el polimorfismo a través la implementación de interfaces, sin embargo, desde Java 8, las interfaces pueden tener métodos **default**.

Aunque son útiles, abusar de ellos para compartir lógica compleja entre interfaces introduce herencia múltiple.

Por ello, cuando necesites compartir comportamiento o estado entre diferentes implementaciones de interfaces, utiliza composición en lugar de heredar comportamiento a través de métodos `default` complejos.

Paquetes

Limita la interfaz publica

Un paquete debe exponer únicamente aquellos tipos que son necesarios para que otros paquetes interactúen con su funcionalidad. Todo lo que no necesites exponer hacia afuera del paquete debe quedar dentro de la interfaz privada del paquete (caja negra).

Esto reduce el acoplamiento y te permite modificar la implementación interna de forma transparente para los usuarios del paquete.

```
package com.example.scheduler;

// al ser publicos, Scheduler y Activity pueden ser importados fuera del package
public class Scheduler {}
public class Activity {}

// al no ser publicos, estas clases son invisibles para el resto del sistema
class Cron {}
class interpreter {}
```

Los cambios de requerimientos no deben propagarse fuera del paquete

Cada paquete debe encapsular sus propias decisiones de diseño. Si un requisito cambia dentro de la responsabilidad de un paquete, el impacto debe limitarse a ese paquete. Los paquetes externos solo dependen de la interfaz pública, y esa interfaz debe ser estable en la medida de lo posible.

Si al modificar un requisito interno te ves obligado a cambiar la interfaz pública de tu paquete, es señal de que no está bien abstraída. Revisa el diseño para separar lo que cambia de lo que permanece estable.

No deben haber dependencias circulares entre paquetes

Las dependencias circulares (paquete **A** depende de **B**, y **B** depende de **A**) generan un alto acoplamiento, dificultan la comprensión del sistema y complica la modularización.

Los paquetes idealmente deberían formar un grafo acíclico dirigido (a.k.a. DAG).

Aplica el principio de inversión de dependencias, los paquetes de alto nivel no deben depender de los de bajo nivel sino que ambos deben depender de interfaces intermedias como DTOs o similares.

Segrega clases no relacionadas

Agrupar en un mismo paquete solo aquellas clases que colaboran estrechamente para cumplir una

única responsabilidad funcional o dicho de otra forma, tienen una alta cohesión. Si alguna de tus clases tiene responsabilidades más bien tangenciales a dicha responsabilidad, muévela a un paquete aislado.

Una buena forma de evaluar la cohesión de un paquete es preguntarte: "Si otro programador abre este paquete, ¿entenderá inmediatamente su propósito?"

Si la respuesta es no, probablemente necesites replantearte el diseño de esa parte del sistema.

El código que define un patrón/arquitectura no debe estar en el mismo paquete que en le que se implementa

El código que define la estructura de un patrón de diseño o la arquitectura de tu sistema no debe estar en el mismo paquete que las implementaciones concretas de ese patrón.

La abstracción en cuestión es por si misma su propia documentación y por lo tanto la implementación en el mismo paquete supondría un detrimento a dicho propósito.

```
com.example.pubsub
  Publisher.java
  Subscriber.java
  Message.java
  Topic.java

com.example.pubsub.kafka
  KafkaPublisher.java
  KafkaSubscriber.java

com.example.pubsub.mosquitto
  MosquittoPublisher.java
  MosquittoSubscriber.java
```

Documentación

Todo paquete que sea ajeno a la arquitectura principal de tu sistema debe incluir un archivo `package-info.java` que describa su propósito y responsabilidad dentro del sistema. Normalmente dichos paquetes están dentro de `utils` o `lib`.

Los paquetes que ya forman parte de la arquitectura no necesitan de dicho archivo pues la arquitectura es por si misma la documentación.

```
/**
 * Provides interfaces and implementations for formatting domain objects
 * into different output representations.
 *
 * The package contains abstractions for text, JSON, XML and custom
 * formatting strategies. Implementations should be stateless and thread-safe
 * whenever possible.
```

```
*/  
package com.example.lib.format;
```

Convención de nomenclatura

Nombres de archivos

Los nombres de archivos se escriben en PascalCase y deben llevar el mismo nombre de la única clase que contienen en singular.

```
# incorrecto  
answerAnalyzer.java  
  
# incorrecto  
Answeranalyzer.java  
  
# incorrecto  
answer-analyzer.java  
  
# incorrecto  
answer_analyzer.java  
  
# correcto  
AnswerAnalyzer.java
```

Nombres de packages

Los nombres de los paquetes (a.k.a. directorios) se escriben todo en minúsculas y sin separación de ningún tipo entre palabras. Para indicar que un paquete está contenido dentro de otro, se usa el punto.

```
// incorrecto  
package com.example.bodyParser;  
  
// incorrecto  
package com.example.Bodyparser;  
  
// incorrecto  
package com.example.BodyParser;  
  
// incorrecto  
package com.example.body-parser;  
  
// incorrecto  
package com.example.body_parser;  
  
// incorrecto
```

```
package com.example.body.parser;

// correcto
package com.example.bodyparser;
```

No uses abreviaturas.

```
// incorrecto
package com.example.ctrls;

// correcto
package com.example.controllers
```

Dicho lo anterior, existen ocasiones en que una abreviación puede a llegar ser más popular y directamente asociable a un significado de lo que es aquello que está abreviando. En dichos casos es correcto usar la abreviación antes que el nombre completo.

```
// correcto
package com.example.io;

// correcto
package com.example.lib;
```

Mantén los nombres consisos y significativos

```
// incorrecto
package com.example.paramresources;
package com.example.resourcemagement;

// correcto
package com.example.resources;
```

Nombres de clases

Las clases se escriben en PascalCase.

```
// incorrecto
class Customerfetcher {}

// incorrecto
class customer_fetcher {}

// incorrecto
class customerfetcher {}

// incorrecto
```

```
class CUSTOMER_FETCHER {}

// correcto
class CustomerFetcher {}
```

Las clases van en singular al representar una entidad, o lo que es lo mismo, una unidad de significado a nivel del dominio del sistema.

```
// incorrecto
public class Addresses {}

// correcto
public class Address {}
```

Dicho esto, si el nombre de tu clase se compone de varias palabras, algunas de ellas pueden estar en plural siempre y cuando el conjunto sea una singularidad.

```
// correcto
public class UserController {}

// también correcto
public class UsersController {}
```

También existen casos en los que por la naturaleza intrínseca de la entidad, el nombre sea un plural que engloba a todo un conjunto de atributos.

```
// correcto
class SystemMetrics {}

// correcto
class UserCredentials {}
```

No se permiten abreviaturas.

```
// incorrecto
class OrdSrv {}

// correcto
class OrderService {}
```

Como se ha mencionado antes, existen abreviaturas que pueden resultar culturalmente más accesibles que la palabra completa. En esos casos es correcto usar la abreviación (en PascalCase) antes que la palabra completa.

```
// incorrecto
```

```

class XMLParser {}

// correcto
class XmlParser {}

// incorrecto
class UniformResourceLocatorGenerator {}

// correcto
class UrlGenerator {}

```

Mantén los nombres de tus clases simples y al punto, evita redundancias o ambigüedades.

```

// incorrecto
class ConnectionMannager {}

// correcto
class Connection {}

// aceptable
class DateTimeUtils {}

// mejor, menos ambiguo
class DateTimeHelpers {}

// aún mejor
class DateTimeAritmetics {}

```

Nombra a tus entidades físicas por lo que son.

```

// incorrecto
public class EnvironmentFileReader {}

// correcto
public class Environment {}

// incorrecto
class ClientData {}

// correcto
class Client {}

// también correcto pues da contexto de su lugar en la arquitectura del sistema
class ClientDto {}

```

Nombra a tus entidades lógicas por lo que hacen.

```
// incorrecto
class PaymentBusiness {}

// correcto
class PaymentProcessor {}

// incorrecto
class EmailUtils {}

// correcto
class EmailSender {}
```

Nombres de interfaces

Al igual que las clases, las interfaces en java representan entidades, y por lo tanto, todas las reglas antes descritas aplican por igual.

Sobre todo no uses una "I" como prefijo para diferenciar entre una clase y una interfaz.

```
// incorrecto
interface IEvent {}

// correcto
interface Event {}
```

Una diferencia significativa al momento de dar nombres a clases e interfaces es que las últimas suelen representar un nivel más alto de abstracción del que lo hacen las clases, de esto surge que las interfaces tiendan a representar no tanto entidades concretas sino roles dentro de la arquitectura y características.

```
// correcto
interface Repository {}
class MeetingRepository implements Repository {}
class RefundRepository implements Repository {}

// correcto
interface Shippable {}
class Docker implements Shippable {}
class Product implements Shippable {}
```

En el caso de interfaces funcionales, nombralas por el rol que cumplen. Dentro, su método debe ser llamado como la verbalización de dicho rol.

```
// correcto
@FunctionalInterface
interface Sorter {
```

```
void sort(List<?> items);  
}
```

Nombres de tipos genéricos

Utiliza nombres cortos y significativos para los parámetros de tipo dentro de una clase genérica. En general se recomienda utilizar una sola letra en mayúscula para representar al tipo, pero si necesitas darle un significado más conciso a nivel de dominio también puedes hacerlo.

Los nombres más comunes para tipos genéricos son los siguientes:

- **T** - Type (tipo general)
- **E** - Element (para elementos de colecciones)
- **K** - Key (para hashes)
- **V** - Value (para hashes)
- **N** - Number (para tipos numéricos)
- **S, U, V**, etc - para múltiples parámetros de tipo

```
// correcto  
public interface List<E> { ... }  
public interface Map<K, V> { ... }  
  
// correcto  
public class Response<T> { ... }  
  
// también correcto  
public class Response<Dto> { ... }
```

Nombres de métodos

Los métodos se escriben en camelCase y deben expresar una acción en relación a la clase.

Nombres de atributos

Los atributos de una clase van en camelCase.

Nombres de propiedades

Las propiedades van en camelCase. Los getters llevan el prefijo **get** y los setters el prefijo **set**.

```
// correcto  
class MusicPlayer {  
    private Song currentSong;  
  
    public Song getCurrentSong() {  
        return currentSong;  
    }  
}
```

```

}

public void setCurrentSong(Song value) {
    currentSong = value;
}
}

MediaPlayer player = new MediaPlayer();
player.getCurrentSong();

```

Existen circunstancias en las que se puede definir un método cuyo único propósito sea modificar un atributo y aún así tenga más sentido a nivel semántico no usar el prefijo **set** pero decir que tal método sigue siendo una propiedad o un setter se convierte en una cuestión de etimología que no afecta al resultado final.

```

// correcto
class MediaPlayer {
    ...
    private boolean lowFidelity = false;

    public void withLowFidelity() {
        lowFidelity = true;
    }
}

MediaPlayer player = new MediaPlayer();
player.withLowFidelity();

```

Nombres de variables

Las variables van en camelCase.

Nombres de constantes

Las constantes van en SCREAM_SNAKE_CASE.

Nombres de enums

Los nombres de los enums van en PascalCase. Los posibles valores del enum van en SCREAM_SNAKE_CASE.

Los nombres de enums no llevan el posfijo **Enum** pues es demasiado redundante.

```

// incorrecto
public enum DayOfWeekEnum { ... }
public enum StatusEnum { ... }

// correcto

```

```
public enum DayOfWeek { ... }  
public enum Status { ... }
```

Evita nombres carentes de significado.

```
// incorrecto  
var tmp = count + aux;
```

Usa el contexto para simplificar el nombre de tu valor

```
// incorrecto  
public class BoardController {  
    private final BoardService boardService;  
    ...  
}  
  
// correcto, se sobre entiende el tipo de entidades con las que la variable 'service'  
va a trabajar.  
public class BoardController {  
    private final BoardService service;  
    ...  
}
```

En el caso de implementaciones de lambdas, es posible reducir el nombre de una variable a un solo caracter ya que su significado se obtiene de el contexto.

```
// correcto  
List<Clients> clients = findClients();  
var maleClients = clients  
    .stream()  
    .filter(client -> client.sex == Sex.MALE)  
    .collect(Collectors.toList());  
  
// mejor, es intuitivo deducir qué significa la variable 'c' en este contexto  
List<Clients> clients = findClients();  
var maleClients = clients  
    .stream()  
    .filter(c -> c.sex == Sex.MALE)  
    .collect(Collectors.toList());
```

is/ can/etc para booleanos

Los métodos que retornan un valor booleano y su propósito es solamente retornar dicho valor deben llevar un nombre que exprese una pregunta que en lenguaje humano se conteste son si o no. Usualmente se usan verbos como "is", "can", "has", "must", ...

```
// incorrecto, no está claro si este método tiene un efecto secundario
private boolean active() {
    ...
}

// correcto
private boolean isActive() {
    ...
}
```

No abreviaciones

No uses abreviaciones. Es preferible un nombre largo pero conciso a un nombre corto pero ambiguo.

No sobreexplicar

Un nombre debe ser lo suficientemente descriptivo sin ser excesivamente largo ni repetir información que ya está presente en el contexto.

Elimina palabras superfluas como **data**, **info**, **object**, **helper**, **manager** cuando no añadan valor real.

Evita la notación Hungara

Cada vez más en desuso, la notación Húngara es una convención de nomenclatura que añade prefijos a los nombres para indicar el tipo de dato (**strName**, **intCount**, **cName**, ...). El uso de esta practica se remonta a los años 80 y 90, cuando las bases de datos no contaban con IDEs robustos. En los tiempos que corren, con los IDEs de programación modernos tal necesidad está por demás rebasada por lo que no es necesario continuar con esta práctica. Esto es especialmente inecesario en lenguajes de tipado fuerte como Java en los cuales el tipo de una variable ya está garantizado desde la compilación.

```
// incorrecto
String strName;
int iCount;
List<Order> lstOrders;
float fPrice;

// correcto
String name;
int count;
List<Order> orders;
double price;
```

Excepciones

Separa el código de control de excepciones del código que puede lanzar la excepción

El bloque `try-catch` debe ser lo más reducido posible, abarcando únicamente el código que puede lanzar la excepción que deseas manejar. Esto mejora la legibilidad y evita capturar excepciones no esperadas en otras partes del bloque.

```
// incorrecto, el try-catch abarca demasiado código
try {
    String input = readInput();           // puede lanzar IOException
    int value = Integer.parseInt(input);  // puede lanzar NumberFormatException
    process(value);                       // puede lanzar otra excepción
} catch (IOException e) {
    ...
}

// correcto
String input;
try {
    input = readInput();
} catch (IOException e) {
    return;
}

int value;
try {
    value = Integer.parseInt(input);
} catch (NumberFormatException e) {
    return;
}

process(value);
```

Aislar el código sensible en bloques `try` pequeños facilita la identificación del origen del error y permite un manejo más granular.

Los mensajes de las excepciones deben ser claros y descriptivos

El mensaje de una excepción debe proporcionar información suficiente para entender qué salió mal y, si es posible, qué se esperaba. Incluye contexto como valores de entrada, identificadores o estado relevante.

No incluyas información sensible (contraseñas, datos personales, etc.) en los mensajes de error, especialmente en logs que puedan ser expuestos.

```
// incorrecto, el mensaje es demasiado genérico
```

```
throw new IllegalArgumentException("Invalid value");

// correct
throw new IllegalArgumentException("El valor '" + input + "' no es un ID válido. Debe ser un número positivo.");
```

Atrapa la excepción más específica primero

Cuando uses múltiples bloques `catch`, coloca las excepciones más específicas antes que las más genéricas.

```
try {
    ...
} catch (FileNotFoundException e) {
    ...
} catch (IOException e) {
    ...
} catch (Exception e) {
    ...
}
```

No lances `Exception` o `Throwable`

Lanzar excepciones genéricas como `Exception` o `Throwable` obliga a los consumidores a capturar una excepción demasiado amplia, ocultando el tipo real de error y dificultando el manejo específico.

```
// incorrecto
public void process() throws Exception { /* ... */ }

// correcto
public void process() throws IOException, ValidationException { /* ... */ }
```

No captures `Exception` o `Throwable`

Capturar `Exception` o `Throwable` es de hecho, peligroso ya que atrapa todas las excepciones, incluidas las no comprobadas (`RuntimeException` y sus subtipos), y puede ocultar errores graves que deberían propagarse, además de que puedes terminar capturando bugs accidentalmente haciendo difícil su identificación y corrección.

```
// incorrecto
try {
    ...
} catch (Exception e) {
    ...
}
```

```
// correcto
try {
    ...
} catch (IOException | SQLException e) {
    ...
}
```

Capturar `Exception` o `Throwable` solo está permitido en el `main thread` de una aplicación para registrar el error antes de terminar, o en frameworks que requieran un manejador global de excepciones. En cualquier otro caso, es una mala práctica.

Si tienes dudas, detén la ejecución

Si no estás seguro de cómo manejar una excepción, o si no hay una acción razonable para recuperarse, no la captures. Deja que la excepción se propague hacia arriba en la pila de llamadas. Esto evita ocultar errores y permite que el sistema falle de forma controlada.

```
// incorrecto
try {
    ...
} catch (IOException e) {
    // No hacer nada o solo loguear y continuar es peligroso
}

// correcto, propaga la excepción si no se puede manejar adecuadamente
public void operacionCritica() throws IOException {
    ...
}
```

Define excepciones que representen reglas de negocio

Crea clases de excepción personalizadas para errores que tengan significado en tu dominio de negocio. Estas excepciones deben reflejar condiciones que son parte de la lógica de la aplicación, no errores técnicos.

```
// correcto
public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String accountId, BigDecimal required,
    BigDecimal available) {
        var message = String.format(
            "Saldo insuficiente en cuenta %s: se requiere %, disponible %s",
            accountId,
            required,
            available
        )
        super(message);
    }
}
```

```
public void withdraw(String accountId, BigDecimal amount) throws
InsufficientBalanceException {
    BigDecimal balance = getBalance(accountId);
    if (balance.compareTo(amount) < 0)
        throw new InsufficientBalanceException(accountId, amount, balance);
    ...
}
```

Atrapa excepciones recuperables

Captura una excepción solo cuando tengas la capacidad de recuperarte del error de manera significativa. Por ejemplo, reintentar una operación, usar un valor por defecto, o mostrar un mensaje al usuario.

```
// correcto
int tries = 3;
while (tries-- > 0) {
    try {
        return remoteCall();
    } catch (ServiceUnavailableException e) {
        if (tries == 0) throw e;
        log.warn("Servicio no disponible, reintentando...");
        Thread.sleep(1000);
    }
}
```

Atrapa excepciones para relanzarlas con un tipo más específico

Cuando captures una excepción técnica y quieras lanzar una excepción de negocio que tenga más sentido para tu dominio, puedes relanzar con un tipo más específico. Siempre que sea posible, encadena la causa original para no perder el stack trace.

```
// correcto
public void pay(PaymentRequest request) throws PaymentFailedException {
    try {
        gateway.charge(request);
    } catch (GatewayException | NetworkException e) {
        throw new PaymentFailedException("Error al procesar el pago", e);
    }
}
```

Atrapa excepciones para guardar un log

Es aceptable capturar una excepción únicamente para guardar un log y luego relanzarla. Asegúrate de incluir el contexto necesario (parámetros, estado) en el log antes de propagar.

```
// correcto
public void analyzeFile(String path) throws IOException {
    try {
        File file = new File(path);
        ...
    } catch (IOException e) {
        log.error("Error al procesar el archivo: {}", path, e);
        throw e;
    }
}
```

Capturar `RuntimeException` es aceptable cuando es lanzado por una biblioteca third-party

Algunas bibliotecas de terceros lanzan `RuntimeException` que representan condiciones de error predecibles. En esos casos, es razonable capturar estas excepciones para manejarlas de forma controlada.

```
// correcto
try {
    JsonObject json = parseJson(input);
} catch (JsonParseException e) {
    log.warn("JSON inválido: {}", input, e);
    return defaultValue;
}
```

Capturar `RuntimeException` es aceptable cuando es un requisito funcional que la app no se detenga

Si el requisito funcional de la aplicación es que no falle completamente ante errores no críticos, puedes capturar `RuntimeException` en puntos apropiados para garantizar la continuidad del servicio. Por ejemplo, un proceso batch que procesa muchos registros; si uno falla, el resto debe continuar.

```
// correcto, si el procesamiento de un record falla, se ignora y la ejecución continua
for (Record record : records) {
    try {
        process(record);
    } catch (RuntimeException e) {
        log.error("Error procesando record: {}", record.getId(), e);
    }
}
```

Esta práctica debe usarse con moderación. Capturar `RuntimeException` debe ser una decisión consciente y justificada por requisitos funcionales, no un comodín para evitar fallos.

Si el `catch` no hace nada con la excepción, añade un comentario justificándolo

Si por alguna razón decides capturar una excepción y no hacer nada con ella, debes incluir un comentario que explique por qué es seguro hacerlo. Esto evita que otro desarrollador elimine el bloque `catch` por error o introduzca un bug al modificarlo.

```
// aceptable, comentario que justifica el catch vacío
try {
    connection.close();
} catch (SQLException e) {
    // The process is finalizing at this point and we don't want
    // this error to stop the shutdown
}
```

Los bloques `catch` vacíos son `code smells`, siempre que sea posible, registra el error o propágalo. Si se decide no hacer nada, la justificación debe ser clara y convincente, y sobre todo evita que la excepción se convierta en regla.

if y else

Usa la técnica del `early return`

Cuando una condición invalida la ejecución de un método, retorna inmediatamente en lugar de envolver todo el código en un bloque `if` anidado. Esto reduce la indentación y hace que el flujo del método sea más lineal y fácil de seguir.

```
// incorrecto
public void process(Order order) {
    if (order != null) {
        if (order.isValid()) {
            ...
        }
    }
}

// correcto
public void process(Order order) {
    if (order == null) return;
    if (!order.isValid()) return;
    ...
}
```

El retorno temprano también se aplica a lanzar excepciones. Si un método recibe parámetros inválidos, lanza la excepción al principio y continúa con la lógica principal sin indentaciones innecesarias.

Evita mutaciones o efectos secundarios en la evaluación de condiciones

Las condiciones en un `if` deben ser expresiones booleanas puras que no modifiquen el estado del programa. Evaluar condiciones con efectos secundarios (como incrementos, asignaciones o llamadas que mutan objetos) hace que el código sea difícil de depurar y predecir.

```
// incorrecto
if (++i > 10) { ... }

// incorrecto
if (list.remove(element)) { ... }

// correcto
i++;
if (i > 10) { ... }

// correcto
boolean removed = list.remove(element);
if (removed) { ... }
```

Este principio también se aplica a las condiciones compuestas con operadores `&&` y `||`. Asegúrate de que ninguna de las sub-expresiones tenga efectos secundarios, ya que el `short-circuit evaluation` puede hacer que dichos efectos no se ejecuten siempre, llevando a comportamientos inesperados.

No uses llaves cuando el bloque de un `if` contiene solo una línea

Cuando el contenido de un `if` se compone de una sola instrucción las llaves son completamente opcionales y por lo tanto no deben incluirse ya que es ruido innecesario entorpeciendo la lectura.

```
// incorrecto
if (condition) {
    doSomething();
}

// correcto
if (condition)
    doSomething();

// mejor
if (condition) doSomething();
```

Suelen darse dos argumentos en contra de esta convención:

- 1.- Es preferible añadir las llaves siempre sin importar cuantas líneas tenga el bloque ya que cuando llegue el momento de refactorizar, no habrá oportunidad para un equivoco si las llaves ya están allí (Es mejor tenerlo y no necesitarlo, que necesitarlo y no tenerlo).
- 2.- Sin las llaves es más difícil leer qué instrucción pertenece o no al bloque, esto es especialmente

confuso en situaciones donde hay muchos niveles de scopes en un espacio muy reducido de código.

Los contra argumentos son los siguientes:

1.- Poner llaves a un `if` de forma profiláctica tiene la intención de eficientar la escritura de código. Sin embargo, si nos remitimos al comienzo de esta guía, la escritura de código es solamente el 20% del tiempo invertido de un programador, el otro 80% es lectura, y por lo tanto es mucho más beneficioso enfocarnos en que el código sea más fácil de leer que de escribir.

2.- Poner llaves para evitar confusiones en partes del código donde hay diferentes niveles de scopes es una forma de solucionar los síntomas sin atacar al problema de raíz. Es decir, que si te cuesta distinguir a qué scope pertenece una línea de código, no es por falta de llaves, sino por que tienes demasiados scopes en el mismo lugar; refactoriza tu código para encapsular los scopes en métodos aislados.

Mueve evaluaciones complejas a métodos independientes

Si una condición es larga, contiene múltiples operadores lógicos o requiere varias líneas para ser comprendida, extráela a un método con un nombre descriptivo. Esto mejora la legibilidad y permite reutilizar la lógica.

```
// incorrecto
if (order != null && order.isActive() && order.getTotal().compareTo(BigDecimal.ZERO) > 0
    && order.getItems().stream().anyMatch(item -> item.getQuantity() > 0)
    && customer != null && customer.isValidAddress()) {
    processOrder(order);
}

// correcto
if (isEligibleForProcessing(order, customer))
    processOrder(order);

private boolean isEligibleForProcessing(Order order, Customer customer) {
    return order != null
        && order.isActive()
        && order.hasPositiveTotal()
        && order.hasItemsInStock()
        && customer != null
        && customer.isValidAddress();
}
```

Evita anidaciones profundas de condicionales

Las anidaciones profundas (más de 2 o 3 niveles) hacen que el código sea difícil de leer y mantener.

Como regla general, si necesitas más de 3 niveles de indentación en una estructura condicional, es una señal de que el código necesita ser refactorizado.

Usa el operador ternario solo si la expresión es simple

El operador ternario (`condición ? valor1 : valor2`) es útil para asignaciones condicionales simples, pero su uso debe limitarse a expresiones cortas y fáciles de entender. Si la expresión ocupa más de una línea o contiene lógica compleja, usa `if`.

```
// incorrecto
String message = user != null && user.isActive() && user.getRole() != null
    ? user.getRole().getWelcomeMessage() != null
    ? user.getRole().getWelcomeMessage()
    : "Bienvenido, " + user.getName()
    : "Bienvenido, invitado";

// correcto
String status = order.isPaid() ? PAID : PENDING;
```

No anides un operador ternario dentro de otro

Anidar operadores ternarios (el llamado "ternario anidado") hace que el código sea extremadamente difícil de leer y propenso a errores de precedencia.

```
// incorrecto
int result = condition1 ? value1 : condition2 ? value2 : condition3 ? value3 :
defaultValue;

// correcto
int result;
if (condition1) result = value1;
else if (condition2) result = value2;
else if (condition3) result = value3;
else result = defaultValue;
```

Separa el código de control de flujo del código de lógica

Las estructuras condicionales deben encargarse únicamente de decidir qué camino tomar, no de ejecutar lógica compleja. La lógica de negocio debe estar en métodos separados.

```
// incorrecto
if (order.isValid()) {
    BigDecimal total = BigDecimal.ZERO;
    for (Item item : order.getItems())
        total = total.add(item.getPrice().multiply(BigDecimal.valueOf(item.
getQuantity())));
    if (total.compareTo(BigDecimal.valueOf(100)) > 0)
        total = total.multiply(BigDecimal.valueOf(0.9));
    order.setTotal(total);
    saveOrder(order);
}
```

```

    sendConfirmationEmail(order);
}

// correcto
public void processOrder(Order order) {
    if (!order.isValid())
        throw new IllegalArgumentException("Invalid order");
    BigDecimal total = calculateTotal(order);
    BigDecimal totalWithDiscount = applyDiscountIfApplicable(total);
    order.setTotal(totalWithDiscount);
    saveOrder(order);
    sendConfirmationEmail(order);
}

private BigDecimal calculateTotal(Order order) { ... }
private BigDecimal applyDiscountIfApplicable(BigDecimal total) { ... }
private void saveOrder(Order order) { ... }
private void sendConfirmationEmail(Order order) { ... }

```

Enums

Usa enums para representar un conjuntos de opciones

Los `enum` son la elección natural cuando un concepto tiene un número finito y conocido de valores posibles en tiempo de compilación. Usarlos, garantiza desde la compilación que una variable solo podrá aceptar un rango de valores previamente pactado a diferencia de usar tipos primitivos.

```

// incorrecto, no es posible controlar qué valor se asigna a nuestra variable
public static final int STATUS_PENDING = 0;
public static final int STATUS_PROCESSING = 1;

int status = 200;

// correcto
public enum OrderStatus {
    PENDING,
    PROCESSING,
    SHIPPED,
    DELIVERED,
    CANCELLED
}

OrderStatus status = OrderStatus.PENDING;

```

Mantén el enum simple

Un `enum` es, idealmente, la representación de un conjunto de valores. Si necesitas añadir múltiples campos, métodos complejos o reglas lógicas, entonces lo que necesitas es una clase.

```
// incorrecto, es demasiado complejo
public enum OrderStatus {
    PENDING,
    PROCESSING,
    SHIPPED,
    DELIVERED,
    CANCELLED;

    public boolean canTransitionTo(OrderStatus target) {
        ...
    }

    public String getDescription(Locale locale) {
        ...
    }

    public void executeAction(Order order) {
        ...
    }
}

// aceptable
public enum Priority {
    LOW(1),
    MEDIUM(2),
    HIGH(3);

    private final int level;

    Priority(int level) {
        this.level = level;
    }

    public int getLevel() {
        return level;
    }
}
```

Usa constantes si lo que te interesa es el valor y no el conjunto

No todos los conjuntos de valores deben ser `enum`. Si lo que necesitas es un valor específico, o si el conjunto de valores puede cambiar en tiempo de ejecución o extenderse por terceros, usa constantes en una clase.

Lambdas

Simple y legibles

Un lambda debe ser corto y expresar claramente su intención en una sola línea si es posible. Si la lógica ocupa más de 3-4 líneas o requiere múltiples instrucciones, considera extraerla a un método con nombre.

```
// incorrecto, el lambda es demasiado complicado
list.stream()
    .filter(item -> {
        if (item.getPrice().compareTo(BigDecimal.ZERO) < 0)
            return false;
        if (item.getQuantity() < 1)
            return false;
        return item.isAvailable();
    })
    .forEach(item -> {
        BigDecimal discounted = item.getPrice().multiply(BigDecimal.valueOf(0.9));
        item.setPrice(discounted);
        System.out.println("Updated: " + item.getName());
    });

// correcto
list.stream()
    .filter(this::isValidItem)
    .forEach(this::applyDiscountAndPrint);

private boolean isValidItem(Item item) {
    return item.getPrice().compareTo(BigDecimal.ZERO) >= 0
        && item.getQuantity() >= 1
        && item.isAvailable();
}

private void applyDiscountAndPrint(Item item) {
    BigDecimal discounted = item.getPrice().multiply(BigDecimal.valueOf(0.9));
    item.setPrice(discounted);
    System.out.println("Updated: " + item.getName());
}
```

Un lambda debe poder leerse como una expresión que describe qué se está haciendo, sin tener que explicar cómo se hace internamente. Si necesitas añadir comentarios en tu lambda, es señal de que deberías extraerlo a un método.

Usa interfaces funcionales estándar

Java ya viene con el paquete de interfaces funcionales `java.util.function` que cubre la mayoría de los problemas con los que te puedas encontrar: `Function`, `Predicate`, `Consumer`, `Supplier`, `UnaryOperator`, `BinaryOperator`, ...

Úsalas en lugar de crear tus propias interfaces funcionales a menos que necesites un nombre

semántico específico.

```
// incorrecto, está reinventando la rueda
@FunctionalInterface
interface StringTransformer {
    String transform(String input);
}

public void process(List<String> list, StringTransformer transformer) { ... }

// correcto
public void process(List<String> list, UnaryOperator<String> transformer) { ... }

process(names, s -> s.toUpperCase());
```

El uso de las interfaces funcionales estándar hace que tu API sea más familiar para otros desarrolladores y permite la composición con métodos como `andThen`, `compose`, `or`, `and` u otros.

Si necesitas un nombre más semántico o con significado a nivel del dominio del sistema, puedes crear una interfaz que extienda de `Function` o `UnaryOperator`.

Implementa lambdas como funciones puras siempre que sea posible

Una función pura es aquella que no tiene efectos secundarios y cuyo resultado depende únicamente de sus argumentos. Tus lambdas deben ser, en la medida de lo posible, funciones puras.

```
// incorrecto
List<String> names = new ArrayList<>();
Stream.of("Pessoa", "Martí", "Neruda")
    .forEach(name -> {
        names.add(name.toUpperCase());
        System.out.println("Processing: " + name);
    });

// correcto
List<String> upperNames = Stream.of("Pessoa", "Martí", "Neruda")
    .map(String::toUpperCase)
    .collect(Collectors.toList());
upperNames.forEach(System.out::println);
```

En ejecuciones en paralelo con `parallelStream()`, los lambdas con mutaciones pueden causar condiciones de carrera y resultados inconsistentes. Siempre prefiere funciones puras cuando uses streams paralelos.

Dicho todo esto, es aceptable que un lambda cuando se usa como `forEach` o `peek`. Sin embargo, incluso en esos casos, es preferible separar la lógica de efectos secundarios del resto de la pipeline.

Tipos Genéricos

Evita hacer casting

Un uso correcto de los genéricos debería eliminar por completo la necesidad de realizar casting en el código. Si te encuentras haciendo casting, es una señal de que estás perdiendo información de tipo o que estás usando tipos raw.

```
// incorrecto
List list = new ArrayList();
list.add("hola");
String s = (String) list.get(0);

// correcto
List<String> list = new ArrayList<>();
list.add("hola");
String s = list.get(0);
```

Evita usar **Object** como tipo genérico

La clase **Object** no pretende ser una alternativa válida a la implementación de genericos y por lo tanto no debe usarse como comodín para almacenar libremente cualquier valor.

Restringe el tipo genérico a un supertipo para acceder a su interfaz pública

Cuando necesites invocar métodos específicos de un tipo, debes restringir el parámetro genérico mediante un **bounded type parameter**.

```
// incorrecto, no sabemos si `T` tiene el método `compareTo()`
public static <T> T max(T a, T b) {
    return a.compareTo(b) > 0 ? a : b;
}

// correcto
public static <T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) > 0 ? a : b;
}
```

Evita usar **wildcards** en los tipos de retorno

Los **wildcards** (**?**, **? extends**, **? super**) en los tipos de retorno obligan a los usuarios de la clase a trabajar con tipos desconocidos, tal y como pasaba con el caso de usar **Object**.

```
// incorrecto
public List<?> getNumbers() { ... }
```

```
// incorrecto
public List<? extends Number> getNumbers() { ... }

// correcto
public List<Number> getNumbers() { ... }

// correcto
class Evaluator<T> {
    public List<T> getElements(T type) { ... }
}

// correcto
class Evaluator<T> {
    public <U> List<U> getElements(Class<U> type) { ... }
}
```

Usa **wildcards** en los parámetros de métodos

Cuando un método solo lee o crea elementos de una estructura genérica, utiliza **wildcards** para aceptar subtipos o supertipos según corresponda.

```
// incorrecto, solo acepta List<Integer>, no List<Number>
public void addIntegers(List<Integer> list) {
    ...
}

// correcto
public double sum(List<? extends Number> numbers) {
    ...
}

// correcto
public void addIntegers(List<? super Integer> list) {
    ...
}
```

Anotaciones

Las anotaciones son metadatos, no lógica de negocio

Las anotaciones deben utilizarse para describir el código, no para implementar comportamiento. La lógica de negocio debe residir en el código de la clase, no en el procesamiento de anotaciones.

Usa anotaciones solo cuando añadan valor

No todas las necesidades requieren una anotación personalizada. Si el mismo propósito puede lograrse con una interfaz, un método o una clase base, prefiere esas opciones antes que crear una

anotación.

Entre las razones que sensatas para usar anotaciones se destacan:

- Para marcar código para procesamiento en tiempo de compilación (`@Override`).
- Para proporcionar configuración declarativa (`@Transactional`).
- Para generar código automáticamente (Lombok).
- Para restricciones de validación (`@NotNull`, `@Size`, ...).

Da valores por default a los atributos de tus anotaciones

Los atributos de una anotación deben tener valores por defecto siempre que sea posible. Esto reduce el boilerplate y permite un uso más limpio de la anotación. Los nombres de los elementos deben ser sustantivos o adjetivos que describan claramente su propósito.

```
// incorrecto
public @interface Audit {
    String action();
    String user();
    boolean sensitive();
}

@Audit(action = "DELETE", user = "system", sensitive = true)
public void deleteData() { ... }

// correcto
public @interface Audit {
    String action();
    String user() default "SYSTEM";
    boolean sensitive() default false;
}

@Audit(action = "DELETE")
public void deleteData() { ... }
```

La convención para el nombre del elemento único es `value()`. Si una anotación tiene un solo elemento, debe llamarse así para permitir la sintaxis abreviada `@Anotacion("valor")` en lugar de `@Anotacion(value = "valor")`.

Especifica siempre `@Retention` y `@Target`

Toda anotación personalizada debe declarar explícitamente su política de retención (`RetentionPolicy`) y los elementos a los que puede aplicarse (`ElementType`). Esto documenta claramente el propósito y el ámbito de la anotación.

```
// incorrecto, no esta claro el ambito de uso de esta anotación
public @interface MyAnnotation { ... }
```

```
// correcto
@Retention(RetentionPolicy.RUNTIME)
@Target({ ElementType.METHOD, ElementType.TYPE })
public @interface Secured {
    String[] roles() default {};
}
```

Usa meta- anotaciones para mejorar el diseño

Las meta- anotaciones son anotaciones que se aplican a otras anotaciones. Úsalas para agrupar funcionalidad o para crear anotaciones compuestas que simplifiquen el uso.

```
// correcto, solo necesita una anotación
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@Transactional
@Audit(action = "MODIFY")
public @interface AuditedTransactional {}

@AuditedTransactional
public void updateOrder(Order order) { ... }
```

Prefiere anotaciones en tiempo de compilación sobre reflexión

Siempre que sea posible, usa anotaciones con `RetentionPolicy CLASS` o `SOURCE` y procesadores de anotaciones en tiempo de compilación. Esto evita el overhead de reflexión en tiempo de ejecución y permite detectar errores en tiempo de compilación.

```
// mejor, procesamiento en tiempo de compilación (Lombok)
@Getter
@Setter
@AllArgsConstructor
public class User {
    private String name;
    private String email;
}

// peor, procesamiento en tiempo de ejecución (reflexión)
@Retention(RetentionPolicy.RUNTIME)
public @interface Validate {
    ...
}
```

Usa anotaciones del JDK para documentar intenciones

Aprovecha las anotaciones estándar del JDK para documentar intenciones y comportamientos:

@Override

Usa `@Override` para indicar que un método sobrescribe un método de superclase o interfaz.

@Deprecated

Usa `@Deprecated` para marcar código que no debe usarse y que solo existe para mantener retrocompatibilidad.

@SuppressWarnings

Usa `@SuppressWarnings` cuando quieras suprimir advertencias del compilador. Debes tener una muy buena razón para usar esta anotación.

@FunctionalInterface

Usa `@FunctionalInterface` para indicar que una interfaz es funcional, idealmente para implementar como lambdas.

@SafeVarargs

Usa `@SafeVarargs` para suprimir advertencias sobre operaciones con varargs y tipos genéricos.

No uses anotaciones para configuración que cambia frecuentemente

Las anotaciones son estáticas; su valor se define en tiempo de compilación. Si una configuración debe cambiar sin recompilar, usa archivos de propiedades, variables de entorno, bases de datos o algún otro recurso externo.

```
// incorrecto, configuración que debería ser dinámica
@Scheduled(cron = "0 9 * * *")
public void dailyReport() { ... }

// correcto
@Scheduled(cron = "${report.cron:0 9 * * *}")
public void dailyReport() { ... }
```

Define anotaciones de forma consistente con el estándar de tu proyecto

Si tu proyecto utiliza un framework (Spring, JPA, JUnit), sigue sus convenciones para anotaciones. No reinventes la rueda; usa las anotaciones proporcionadas por el framework antes de crear las tuyas propias.

Paralelismo

Diseña los procesos de forma aislada

Cada tarea que se ejecuta en paralelo debe ser independiente de las demás en cuanto a su lógica y datos. El objetivo es que no haya dependencias entre tareas que puedan causar colisiones o sincronización.

```
// correcto
List<Order> orders = getOrders();

List<CompletableFuture<Report>> futures = orders.stream()
    .map(order -> CompletableFuture.supplyAsync(() -> processOrder(order)))
    .collect(Collectors.toList());

List<Report> reports = futures.stream()
    .map(CompletableFuture::join)
    .collect(Collectors.toList());
```

Si una tarea necesita el resultado de otra para continuar, entonces no son tareas independientes. En ese caso, el paralelismo no es apropiado, o debes diseñar un flujo de trabajo con dependencias explícitas.

Evita compartir memoria

El estado compartido (variables mutables accesibles desde múltiples hilos) dispara la complejidad del trabajo con multihilos. Siempre que sea posible, cada tarea debe trabajar con sus propios datos locales, sin acceder a variables compartidas.

```
// incorrecto, increment no es un método 'thread safe'
private int counter = 0;

public void increment() {
    counter++;
}

// correcto, cada hilo tiene su propia variable
public int process(Order order) {
    int localCounter = 0;
    ...
    return localCounter;
}
```

...incluso si es de solo lectura

Aunque una variable compartida sea de solo lectura, compartir memoria implica que todos los hilos acceden a la misma ubicación de memoria. Esto puede causar problemas de **cache coherence** y afectar al rendimiento. Además, si el hilo principal o un proceso externo muta el valor, el riesgo aumenta. De ser posible usa copias o datos inmutables en cada hilo.

```
// correcto
public class ProcessTask implements Runnable {
    private final ImmutableData data;

    public ProcessTask(ImmutableData data) {
```

```

    this.data = data;
}

@Override
public void run() {
    ...
}
}

```

Si compartir memoria es inevitable, usa un sistema de semáforos

Cuando no puedas evitar el estado compartido, utiliza mecanismos de sincronización adecuados: `synchronized`, `Lock`, `Semaphore` o estructuras concurrentes como `ConcurrentHashMap`.

Las estructuras del paquete `java.util.concurrent` están optimizadas para entornos de alta concurrencia. Siempre que sea posible, usa `ConcurrentHashMap`, `ConcurrentLinkedQueue`, `Atomic*`, etc., en lugar de sincronizar manualmente.

```

// correcto, usa un 'Lock' para proteger el estado compartido
private final Lock lock = new ReentrantLock();
private int sharedCounter = 0;

public void increment() {
    lock.lock();
    try {
        sharedCounter++;
    } finally {
        lock.unlock();
    }
}

// correcto, usa 'AtomicInteger' (sin bloqueo)
private final AtomicInteger atomicCounter = new AtomicInteger(0);

public void increment() {
    atomicCounter.incrementAndGet();
}

```

...o arquitecturas reactivas

Las arquitecturas reactivas (como Project Reactor, RxJava) tratan los flujos de datos asíncronos como streams. Esto promueve la inmutabilidad y evita el estado compartido al pasar datos a través de canales.

...o sistemas basados en pub/sub

Las arquitecturas `pub/sub` desacoplan productores y consumidores. Cada mensaje se procesa de forma independiente, y los componentes no comparten estado, comunicándose únicamente a

través de mensajes.

```
// correcto
public class OrderPublisher {
    private final ExecutorService executor = Executors.
newVirtualThreadPerTaskExecutor();
    private final List<Consumer<Order>> subscribers = new CopyOnWriteArrayList<>();

    public void publish(Order order) {
        for (var subscriber : subscribers)
            executor.submit(() -> subscriber.accept(order));
    }
}
```

Usa VirtualThreads

Los **virtual threads** permiten crear millones de hilos ligeros sin el overhead de los hilos del sistema operativo. Esto hace que el código concurrente sea más sencillo de escribir y escalable. Usa `Executors.newVirtualThreadPerTaskExecutor()` para crear un executor que lance un nuevo `VirtualThread` por cada tarea.

```
// correcto
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    List<CompletableFuture<Report>> futures = orders
        .stream()
        .map(order -> CompletableFuture.supplyAsync(
            () -> processOrder(order), executor))
        .collect(Collectors.toList());

    List<Report> reports = futures
        .stream()
        .map(CompletableFuture::join)
        .collect(Collectors.toList());
}
```

Ten en cuenta que los **virtual threads** trabajan internamente con los **Threads** clásicos de Java, por lo que no son ni un sustituto ni una optimización. Concretamente los casos de uso donde se pueden destacar es cuando necesitas manejar de forma concurrente muchas tareas aisladas, de cortos ciclos de vida y con tiempos de espera prolongados.

Imports

No uses wildcard imports

Los imports con wildcard (*) importan todas las clases de un paquete ya que ocultan las dependencias reales del archivo y pueden provocar conflictos de nombres inesperados. Tampoco queda claro qué clases del paquete se están utilizando realmente, si una clase se elimina o

renombrar, el wildcard sigue siendo válido, dificultando la detección de dependencias rotas.

```
// incorrecto
import java.util.*;
import java.io.*;

// correcto
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.io.InputStream;
import java.io.OutputStream;
```

Menos de 20 imports por archivo

Un archivo fuente con demasiados imports es una señal de que la clase tiene demasiadas dependencias y probablemente estas cayendo en el antipatrón del **God Object**, o lo que es lo mismo, estas rompiendo el principio single responsibility. Considera en su lugar refactorizar la clase en múltiples clases más cohesivas.

```
// incorrecto, demasiadas dependencias
import com.example.auth.*;
import com.example.orders.*;
import com.example.customers.*;
import com.example.inventory.*;
// ... 20 más
```

Si un archivo supera los 20 imports, es síntoma de que debes replantearte el diseño de tu clase.

Organización de imports

Los imports deben organizarse de forma consistente para que sean fáciles de leer y mantener. El orden es el siguiente:

- 1.- Imports estándar de Java (java.* y javax.*) en orden alfabético.
- 2.- Imports de terceros en orden alfabético.
- 3.- Imports de tu propio proyecto en orden alfabético.
- 4.- Imports estáticos.

Cada uno de estos apartados va separado por una línea en blanco de separación.

```
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Collections;
```

```
import java.util.List;

import org.apache.commons.lang3.StringUtils;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import com.example.core.Entity;
import com.example.orders.OrderStatus;

import static java.util.stream.Collectors.toList;
import static org.junit.jupiter.api.Assertions.assertEquals;
```

No incluyas imports para elementos usados solo en los Javadocs

Los enlaces a clases en los Javadocs ([@link](#), [@see](#), etc.) no necesitan ser importados. Java permite usar nombres completamente cualificados en los Javadocs sin necesidad de declarar un import. Incluir esos imports añade ruido innecesario al archivo.

```
// incorrecto
import com.example.orders.OrderService;

/**
 * Process an order using the class {@link OrderService}.
 */
public class OrderProcessor { ... }
```

Uso de imports estáticos

Los imports estáticos permiten acceder a miembros estáticos (métodos o campos) sin calificarlos con el nombre de la clase.

```
import static java.lang.Math.PI;
import static java.util.stream.Collectors.toList;

public class CircleAreaTest {
    void testArea() {
        double radius = 5.0;
        double area = PI * radius * radius;
        ...
        List<String> names = getNames()
            .stream()
            .filter(n -> n.startsWith("A"))
            .collect(toList());
    }
}
```

Testing

Convención de nombres para los tests

Los nombres de los métodos de test deben ser descriptivos y reflejar el escenario que están probando. La convención más utilizada es el formato **GivenWhenThen** o una variante que indique claramente qué se prueba y qué se espera.

En general el nombre del método de test debe ser suficiente para entender qué se está probando sin necesidad de leer la implementación.

```
// correcto, usando la estructura Given_When_Then
@Test
void givenValidOrder_whenProcess_thenOrderIsSaved() {
    Order order = new Order("123", BigDecimal.TEN);
    orderService.process(order);
    assertThat(order.getStatus()).isEqualTo(Status.PROCESSED);
}

// también correcto, usando la estructura should_When
@Test
void shouldThrowException_whenOrderIsNull() {
    assertThatThrownBy(() -> orderService.process(null))
        .isInstanceOf(IllegalArgumentException.class);
}
```

Enfoca los tests a los requerimientos, no al coverage

El objetivo de los tests no es alcanzar un porcentaje de cobertura arbitrario (¿80%? ¿100%?), sino verificar que el código cumple con los requisitos funcionales y no funcionales del sistema. Céntrate en probar los casos de uso y los límites del dominio, no en cubrir cada línea de código (en todo caso, eso debería ser un efecto colateral).

También hay que tener presente que cubrir el 100% del código no garantiza que tu sistema funcione correctamente. Un test que no verifica un comportamiento real es ruido que dificulta el mantenimiento. Prioriza la calidad y relevancia de los tests sobre la cantidad.

Pruebas pequeñas, específicas y deterministas

Cada test debe verificar una única cosa y hacerlo de forma determinista. Un test que falla intermitentemente o que depende del orden de ejecución es un problema grave que debe corregirse inmediatamente.

```
// incorrecto, el test verifica demasiadas cosas
@Test
void testOrderProcessing() {
    Order order = new Order();
    order.addItem(new Item("Laptop", 1));
}
```

```

orderService.validate(order);
orderService.applyDiscount(order);
orderService.save(order);
assertThat(order.getStatus()).isEqualTo(Status.COMPLETED);
assertThat(order.getTotal()).isLessThan(1000);
}

// correcto
@Test
void whenOrderHasOneItem_thenTotalIsCorrect() {
    Order order = new Order();
    order.addItem(new Item("Laptop", 1));
    assertThat(order.getTotal()).isEqualTo(1000);
}

@Test
void whenOrderContainsValidItems_thenStatusIsCompleted() {
    Order order = new Order();
    order.addItem(new Item("Laptop", 1));
    orderService.process(order);
    assertThat(order.getStatus()).isEqualTo(Status.COMPLETED);
}

```

Los tests deben permanecer aislados

Un test no debe depender de la ejecución de otros tests. Cada test debe poder ejecutarse de forma independiente y en cualquier orden.

Testea Singletons con inyección de dependencias

Los Singletons dificultan los tests porque mantienen estado global. Si necesitas testear un Singleton, asegúrate de que su estado pueda ser reseteado entre tests mediante inyección de dependencias o proporciona un método de reset para los tests.

```

// correcto
public class ConfigManager {
    private static ConfigManager instance;
    private final Properties properties;

    private ConfigManager(Properties properties) {
        this.properties = properties;
    }

    public static synchronized ConfigManager getInstance(Properties properties) {
        if (instance == null)
            instance = new ConfigManager(properties);
        return instance;
    }
}

```

```

public static synchronized void resetInstance() {
    instance = null;
}

@BeforeEach
void setup() {
    ConfigManager.resetInstance();
}

@Test
void whenConfigManagerIsInitialized_thenPropertiesAreLoaded() {
    Properties testProps = new Properties();
    testProps.setProperty("key", "value");
    ConfigManager config = ConfigManager.getInstance(testProps);
    assertThat(config.getProperty("key")).isEqualTo("value");
}

```

Usa tests parametrizados para múltiples casos

Los tests parametrizados reducen la duplicación de código cuando necesitas probar el mismo comportamiento con diferentes entradas y salidas esperadas.

```

// correcto
@ParameterizedTest
@CsvSource({
    "2, 3, 5",
    "0, 0, 0",
    "-1, 1, 0",
    "100, -50, 50"
})
void whenAddingNumbers_thenResultIsCorrect(int a, int b, int expected) {
    Calculator calculator = new Calculator();
    assertThat(calculator.add(a, b)).isEqualTo(expected);
}

```

Una suite de tests por clase

Cada clase de producción debe tener su correspondiente clase de test facilitando así la localización de los tests y mantiene la relación uno a uno entre código y pruebas. Los tests de integración pueden tener su propia estructura, pero la regla general es una clase de test por clase de producción.

Usa mocks antes que fake classes

Para aislar cada unit test, utiliza mocks en lugar de crear implementaciones falsas de dependencias. Los mocks son más flexibles, permiten verificar interacciones y son más fáciles de mantener que clases fakes escritas a mano.

Haz **Assert** solo sobre lo que estás testeando

Un test debe contener únicamente aserciones que verifiquen el comportamiento específico que está probando. Evita añadir aserciones adicionales que no estén directamente relacionadas con el escenario del test.

```
// incorrecto, aserciones no relacionadas
@Test
void whenOrderIsValid_thenProcessed() {
    Order order = new Order("123", BigDecimal.TEN);
    orderService.process(order);
    assertThat(order.getStatus()).isEqualTo(Status.PROCESSED);
    assertThat(order.getTotal()).isGreaterThan(BigDecimal.ZERO);
    assertThat(order.getItems()).isEmpty();
}

// correcto
@Test
void whenOrderIsValid_thenStatusIsProcessed() {
    Order order = new Order("123", BigDecimal.TEN);
    orderService.process(order);
    assertThat(order.getStatus()).isEqualTo(Status.PROCESSED);
}
```

Comentarios

Escribir código es como contar un chiste. Si es bueno no necesita que lo expliques.

No añadas comentarios en tu código e ignora el resto de esta sección.

Refactoriza en vez de comentar

Si sientes que tu código es demasiado complejo como para ser entendido por otros programadores (por ejemplo, tú mismo dentro de 2 semanas), es natural querer tener la cortesía de documentar la intención de dicho código. Sin embargo, te recomiendo que antes de hacerlo te detengas por un momento para replantearte si es posible reducir la complejidad de tu código.

```
// incorrecto, el método es tan complejo que hace falta comentar cada línea
public void processOrder(Order order) {
    // 1.- Validations
    if (order == null)
        throw new IllegalArgumentException("El pedido no existe");
    if (order.getItems() == null || order.getItems().isEmpty())
        throw new IllegalArgumentException("El pedido necesita por lo menos un articulo");
    if (order.getCustomer() == null)
        throw new IllegalArgumentException("Se necesita un clienete");
}
```

```

// 2.- Calc of the total
BigDecimal total = BigDecimal.ZERO;
for (Item item : order.getItems())
    total = total.add(item.getPrice().multiply(BigDecimal.valueOf(item.
getQuantity())));
order.setTotal(total);

// 3.- Apply the discount if is a big quantity
if (total.compareTo(BigDecimal.valueOf(1000)) > 0) {
    BigDecimal discount = total.multiply(BigDecimal.valueOf(0.10));
    total = total.subtract(discount);
    order.setTotal(total);
}

// 4.- Saving in the database
orderRepository.save(order);

// 5.- Sending an email notification
emailService.sendConfirmation(
    order.getCustomer().getEmail(),
    "Order " + order.getId() + " processed"
);
}

// correcto, el código es su propia documentación
public void processOrder(Order order) {
    validateOrder(order);
    calculateTotal(order);
    applyDiscountIfApplicable(order);
    saveOrder(order);
    sendConfirmationEmail(order);
}

private void validateOrder(Order order) { ... }
private void calculateTotal(Order order) { ... }
private void applyDiscountIfApplicable(Order order) { ... }
private void saveOrder(Order order) { ... }
private void sendConfirmationEmail(Order order) { ... }

```

Usa JavaDocs

Usa la sintaxis de JavaDocs para documentar tu código, pues está ofrece una estructura ya conocida y compartida por todo desarrollador de Java.

Además esta estructura es usada por la mayoría de los IDEs y editores de código modernos para ofrecer ayuda contextual mientras dichas clases y métodos son utilizados.

Otra característica es que pueden utilizarse otras herramientas para convertir dichos comentarios en documentos (pdf, html, json...).

No "guardar" código como comentario

Todo aquel código que no se ejecute, y en general, que no sea necesario; debe ser eliminado.

Si quieres preservar dicho código porque sospechas que en algún futuro vas a necesitarlo, git ya se encargara de mantener dicho código en su historial de cambios.

Comentarios declarativos

Los comentarios, al igual que el código, deben describir qué es lo que el código hace y no cómo lo hace.

Solo comenta la interfaz pública

Ya que el objetivo del comentario es ayudar a otros programadores a entender como usar un fragmento de código, no hace falta dar información sobre su interfaz privada.

```
/** Inspects pdfs to validate if they have the right format for the autocompletion */
public class PdfAnalyzer {
    /** Starts the analyze process once it validates the path is correct */
    public PdfFile load(String path) { ... }

    private void readFile() { ... } // <--- Al ser privado no hace falta explicar que
    hace
}
```

Explica código de nicho

Existe ocasionalmente la circunstancia en que un fragmento de código representa a una entidad que no forma parte directa de los requerimientos del proyecto (y que por lo tanto no tiene documentación a nivel de negocio), y que además no forma parte de la cultura general, sino que es conocimiento de nicho y que puede ser muy técnico o muy complejo.

En tales ocasiones es deseable que el programador se explique sobre el funcionamiento interno de dicho código. Incluso si es posible, añadiendo enlaces externos a otras fuentes de información.

```
/**
Fourier transform is based on the Fourier series that represents periodic functions as
an infinite sum of sines and cosines. This kind of decomposition is possible due to
orthogonality properties of sine and cosine functions. The Fourier transform is a
generalization of complex Fourier series in the limit as the period approaches
infinity.

Fourier transformation is calculated using the discrete Fourier transform (DFT) that
inherently assumes that the input signal is periodic and spectral resolution of the
transformation is determined by the sampling step and the number of sample points. If
the amplitude of the input signal starts from zero and ends at zero, spectral
resolution can be increased adding zero sample points to the end or beginning of the
```

```
signal increasing the number of sample points. This operation is commonly known as zero padding.
```

```
https://en.wikipedia.org/wiki/Fourier_transform
*/
public class FourierTransform {
    // ...
}
```

Etiquetas

Idealmente el programador debería aspirar a seguir la "filosofía del boy scout", que consiste en que: Aquel lugar al que vayas, haz de dejarlo, no como estaba antes de que llegases, sino un poco mejor.

En la práctica esto no siempre es posible. Para estas situaciones, es posible dejar comentarios para llamar la atención de otros programadores sobre alguna parte del programa.

Considera también la posibilidad de usar otras vías de comunicación para esto, como un correo, jira, o incluso comunicación cara a cara.

TODO

Usa TODO para denotar funcionalidades pendientes de ser implementadas.

```
public void lowerRank(Product product) {
    // TODO: We are waiting for business to define this rule.
}
```

FIXME

Usa FIXME para indicar que una parte del programa tiene fallos y debe ser corregida.

```
// FIXME: This method returns the same order twice in a row sometimes.
public List<?> shuffle(List<?> items) {
    ...
}
```

OPTIMIZE

Usa OPTIMIZE para denotar un funcionamiento lento o ineficiente que puede ocasionar problemas de performance.

```
// OPTIMIZE: This implementation has a O(n^2) complexity but it could be just O(n)
public boolean hasDuplicates(List<Customer> customers) {
    ...
}
```

HACK

Usa HACK ante un "code smell", es decir, una implementación de código con prácticas cuestionables que debería ser refactorizado.

```
// HACK: Use the Jackson library instead of a manual parsing
public String asJson(Quotation quotation) {
    ...
}
```

REVIEW

Usa REVIEW cuando necesites una confirmación de que una sección de código está funcionando de la forma esperada por los usuarios.

```
// REVIEW: Are we sure this is the way the clients wants to authenticate?
public boolean login(String phoneNumber, String password) {
    ...
}
```

Café para los más cafeteros

La siguiente sección no es necesaria, puedes ignorarla.

Las siguientes recomendaciones son más restrictivas que todo lo antes descrito, y por lo tanto, existe una mayor probabilidad de que encuentres escenarios en tu día a día en los que tenga más sentido ignorarlas.

Sin embargo, te recomiendo probar a trabajar bajo estas restricciones; por lo menos durante un tiempo. Del hecho de trabajar de una forma tan restrictiva, estarás obligado a pensar "fuera de la caja" para conseguir tus objetivos.

- No más de 100 líneas de código por archivo (contando los imports pero no los comentarios).
- No más de 12 líneas de código por método (no, no puedes poner más de una instrucción en la misma línea).
- No más de 15 imports por archivo (no, no puedes usar wildcard imports).
- No más de 3 niveles de indentación por método.
- No más de 4 parametros por método (si necesitas más argumentos, significa que hay una entidad que no estas definiendo).